

**SMART SCHOOL BUS SYSTEM WITH AUTOMATED
ATTENDANCE AND ENVIRONMENT MONITORING**

HEMA A/P SAUNDARASAN

52224123036

**MALAYSIAN INSTITUTE OF INFORMATION
TECHNOLOGY**

UNIVERSITI KUALA LUMPUR

OCTOBER 2025

**SMART SCHOOL BUS SYSTEM WITH AUTOMATED
ATTENDANCE AND ENVIRONMENT MONITORING**

HEMA A/P SAUNDARASAN

52224123036

**Report Submitted to Fulfil the Requirements
For the Bachelor of Information Technology (Hons) in
Internet of Things
Malaysian Institute of Information Technology,
Universiti Kuala Lumpur**

OCTOBER 2025

DECLARATION

I hereby declare that this project report is based on my original work except for citations and quotation which have been duly acknowledged. I also declare that it has not been previously and concurrently submitted for any other degree or award at Universiti Kuala Lumpur Malaysian Institute of Information Technology (MIIT) or other institutions. The content of this report is the result of my own efforts under the guidance of my supervisor, and any assistance received has been duly recognized.

Date : 18/06/2026

Signature: 

Full Name: Hema A/P Saundarasan

ID No: 52224123036

APPROVAL PAGE

We have examined this dissertation and verified that it meets the program and institute's requirements for the Bachelor of Information Technology (Hons) in Internet of Things.

Date : _____

Signature:

Supervisor: Madam Faridah Binti Yahya

Original Stamp:

Date : _____

Signature:

Accessor 1: Dr Nur Farahwahida Ab Aziz

Original Stamp:

Date : _____

Signature:

Accessor 2 : Sir Azmi Ahmad

Original Stamp:

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to God for endless blessings, guidance, strength, and grace throughout my academic journey. There were many moments of difficulty, uncertainty, and exhaustion during the completion of this Final Year Project, and it is through the guidance that I was able to persevere and successfully complete this study. I am truly thankful for the opportunities and strength given to me throughout this journey.

I would like to extend my heartfelt appreciation to my beloved parents and my sisters for their unconditional love, patience, encouragement, and unwavering support throughout my studies. Thank you for believing in me and motivating me to achieve my goals. Your sacrifices, understanding, and continuous support have been my greatest source of inspiration and strength. I am deeply grateful for everything you have done for me.

I would also like to express my sincere gratitude to my supervisor, Ts Faridah Binti Yahya, for her continuous guidance, encouragement, and valuable feedback throughout the completion of this project. The improvement and success of this project have been much aided by her support and insightful advice. I also wish to express my gratitude to the project accessor, Dr Nur Farahwahida, for her time, analysis, and useful feedback throughout the assessment. Her ideas have improved the quality of this project. My second accessor, Sir Azmi Ahmad, for his time, idea and guidance. His ideas and feedback elevated my project quality and scope.

My friends and classmates are much appreciated for their insightful conversations, encouraging support, and helpful criticism throughout the research. Last but not least, I would like to acknowledge myself for having the determination, perseverance, and resilience to complete this Final Year Project.

This journey was filled with challenges that tested my patience, confidence, and commitment. However, I remained focused on my goals and continued to learn and grow from every experience.

To everyone who has contributed directly or indirectly to the completion of this project, I extend my sincere appreciation and gratitude. Thank you for your support, encouragement, and contributions in making this achievement possible.

LIST OF TABLES

Table 1: A comparison table of existing systems.....	28
Table 2: Cost of the project.....	41
Table 3: List of hardware components and purposes.....	42
Table 4: List of software components and purposes.....	43
Table 5: Input hardware devices and the function.....	46
Table 6: Output hardware devices and their function.....	49
Table 7: Hardware devices connection table	52
Table 8: RFID Attendance Testing Results.....	70
Table 9: Driver Authentication Testing Results.....	71
Table 10: Temperature Sensor Testing Results.....	72
Table 11: Gas Sensor Testing Results.....	73
Table 12: Alert System Testing Results.....	74
Table 13: Cloud Data Upload Testing Results.....	74
Table 14: Web Dashboard Monitoring Testing Results.....	75
Table 15 Lists the project Objectives and Achievement Status.....	78

LIST OF FIGURES

Figure 2.2.2: Simulated prototype of Environmental Monitoring System.....	22
Figure 2.2.3: Interface of IoT-Based Smart School Bus System.....	24
Figure 2.2.4: System setup of Secured College Bus System.....	24
Figure 3.1: Block diagram.....	30
Figure 3.2: Wokwi Project Circuit Diagram	31
Figure 3.3: Wokwi project dashboard.....	32
Figure 3.4: Project flowchart.....	33
Figure 3.5: Wokwi Simulation's Flowchart.....	34
Figure 3.6: Waterfall model methodology.....	37
Figure 4.1 ESP32 Development Board.....	45
Figure 4.2 MFRC522 RFID Reader Module.....	47
Figure 4.3 DHT11 Temperature and Humidity Sensor.....	47
Figure 4.4 MQ2 Gas Sensor.....	48
Figure 4.5 Push Buttons for Route Selection.....	48
Figure 4.6 16×2 LCD Display.....	49
Figure 4.7 OLED Display Module.....	50
Figure 4.8 Servo Motor.....	50
Figure 4.9 Buzzer Module	51
Figure 4.10 Complete Virtual Circuit Diagram.....	52

Figure 4.11 Right Side view of the prototype created.....	53
Figure 4.12 Front view of the prototype created.....	54
Figure 4.13 Left Side view of the prototype created.....	54
Figure 4.14 Back view of the prototype.....	54
Figure 4.15 Data Flow Architecture (ESP32 to Cloud APIs).....	56
Figure 4.16 ThingSpeak Channels.....	57
Figure 4.17 Real-time sensor data stored in ThingSpeak Platform.....	58
Figure 4.18 Channel settings of Environment Monitoring Channel.....	58
Figure 4.19 The upload_environment function in main.py.....	59
Figure 4.20 Real-time sensor data stored in ThingSpeak Platform.....	59
Figure 4.21 Channel settings of Attendance Monitoring Channel.....	60
Figure 4.22 The upload_attendance function in main.py.....	60
Figure 4.23 Authorized RFID Database.....	62
Figure 4.24 Student Information Database.....	63
Figure 4.25: The files used to create the web dashboard development...	65
Figure 4.26 Smart School Bus's Web Dashboard.....	65
Figure 4.27 Telegram Bot API linked with a Telegram group chat.....	67
Figure 4.28 Telegram Attendance Notification.....	67
Figure 4.29 Telegram Duplicate entries Notification.....	68
Figure 4.30 Telegram Unauthorized Access Notification.....	68
Figure 4.31 Telegram Ride Started Notification.....	68
Figure 4.32 Telegram Ride Summary Notification.....	69
Figure 4.33 Valid Student RFID Scan.....	70
Figure 4.34 Duplicate RFID Detection.....	70

Figure 4.35 Unauthorized RFID Detection.....	71
Figure 4.36 Successful Driver Authentication.....	71
Figure 4.37 Invalid Driver Card Detection.....	71
Figure 4.38 Coding of High Temperature function.....	71
Figure 4.39 High Temperature Alert.....	72
Figure 4.40 Coding of High Gas Level function.....	73
Figure 4.41 High Gas Level Alert.....	73
Figure 4.42 Serial monitor Output on Environment Data Upload.....	74
Figure 4.43 Serial Monitor Output on Ride Summary Data Upload.....	75
Figure 4.44 Serial Monitor Output on API.....	75
Figure 4.46 Dashboard Attendance Update.....	76
Figure 4.47 Figure 4.47 Sensor graph display.....	76

LIST OF ABBREVIATIONS

Abbreviation	Description
IoT	Internet of Things
RFID	Radio Frequency Identification
GPS	Global Positioning System
AI	Artificial Intelligence
HTTP	Hypertext Transfer Protocol

ABSTRACT

School transportation systems often depend on manual attendance recording and lack continuous monitoring of environmental conditions, which may compromise student safety and operational efficiency. To address these limitations, this project presents a Smart School Bus System with Automated Attendance and Environmental Monitoring based on Internet of Things (IoT) technology. The proposed system utilizes an ESP32 microcontroller integrated with environmental sensors such as DHT11 for temperature and humidity monitoring and MQ2 for gas detection, along with attendance input devices for student identification. Real-time data is transmitted to a cloud-based dashboard developed using the ThingSpeak platform, enabling remote monitoring, alert notifications, and basic system interaction through a web dashboard. The system design and functionality were validated through virtual circuit simulation before hardware integration. Emphasis is placed on low-cost implementation, ease of use, and suitability for mobile environments. The results indicate that the proposed system can improve attendance accuracy, enhancing environmental safety, and supporting real-time decision-making in school bus operations.

TABLE OF CONTENTS

DECLARATION	iii
APPROVAL PAGE.....	iv
ACKNOWLEDGEMENT	v
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	xi
ABSTRACT	xii
TABLE OF CONTENTS.....	xiii
CHAPTER 1: INTRODUCTION	16
1.1 Introduction	16
1.2 Problem Statement	17
1.3 Project Objectives	17
1.4 Project Scope.....	18
1.5 Significance of the project.....	18
1.6 Expected Outcome	18
1.7 Project Limitation	19
CHAPTER 2: LITERATURE REVIEW	20
2.1 Introduction	20
2.2 Review of Existing System.....	20
2.2.1 IoT-Based Intelligent Attendance System	21
2.2.2 Environmental Monitoring in Bus Transportation.....	22
2.3.3 IoT-Based Smart School Bus Monitoring and Notification System.....	23
2.2.4 Secured College Bus Management System Using IoT for COVID-19 Screening.....	24
2.2.5 Face Recognition Attendance System Based on Real-time Video Processing	26
2.3 Comparison between Similar Existing Systems and the Proposed System	28
2.4 Limitation.....	29
CHAPTER 3: METHODOLOGY	30
3.1 Introduction	30
3.2 System Overview	30
3.2.1 Block Diagram.....	30

3.2.2 Circuit Diagram	31
3.2.3 Flowchart	33
3.2.4 Control modes of the system	35
3.2.5 Push Notifications	36
3.3 The Methodology Model.....	36
3.3.1 Requirement Phase	37
3.3.2 Design Phase.....	38
3.3.3 Implementation Phase	38
3.3.4 Testing Phase	39
3.3.5 Deployment and Evaluation Phase	40
3.3.6 Maintenance Phase	40
3.4 Cost of the Project	41
3.5 System Requirement Analysis	42
3.5.1 Hardware Requirement.....	42
3.5.2 Software Requirement	42
CHAPTER 4: RESULTS AND DISCUSSION	44
4.1 Introduction	44
4.2 Hardware Implementation	44
4.2.1 Microcontroller Configuration	45
4.2.2 Input Devices	46
4.2.3 Output Devices	49
4.2.3 Circuit Design and Connection.....	51
4.2.4 Prototype Development	53
4.3 Software Development.....	55
4.3.1 System Flow Implementation.....	55
4.3.2 API Design and Development	56
4.3.3 ThingSpeak Cloud Platform Integration	57
4.3.4 Database Design and Development	62
4.3.4 Web Dashboard Development.....	65
4.3.6 Telegram Notification Integration.....	67
4.4 System Testing and Results.....	69
4.4.1 RFID Attendance Testing	70
4.4.2 Driver Authentication Testing.....	71

4.4.3 Temperature Sensor Testing	72
4.4.4 Gas Sensor Testing.....	72
4.4.5 Alert System Testing.....	73
4.4.6 Cloud Data Upload Testing	74
4.4.7 Dashboard Monitoring Testing	75
4.5 Discussion.....	76
4.5.1 System Performance	76
4.5.2 Achievement of Project Objectives.....	77
4.5.3 Limitations of the system	79
CHAPTER 5: CONCLUSIONS AND RECOMMENDATIONS.....	80
5.1 Introduction	80
5.2 Conclusion	80
5.3 Future Recommendations.....	82
5.4 Chapter Summary.....	82
REFERENCES.....	84
APPENDIX A: SOURCE CODE	1
Main.py	2
Connect_wifi.py.....	13
I2c_lcd.py.....	13
Ssd1306.py.....	14
Servo.py.....	14
Mfrc522.py	15
APPENDIX B: WEB DASHBOARD CODING	16
Index.html	17
Script.js	17
Style.css	18
APPENDIX C: SOFTWARE PLATFORMS.....	19
ThingSpeak.....	20
Web Dashboard	21
Telegram	22
APPENDIX D: GANTT CHART	23

CHAPTER 1: INTRODUCTION

1.1 Introduction

In recent years, technological innovation has become an essential component of improving daily life activities, particularly in education and transportation. Using the Internet of Things (IoT) has turned conventional systems into intelligent, networked platforms able to perform real-time monitoring and automatic replies. School transportation is one sector that could be much improved by adopting the Internet of Things. For parents, educators, and schools, the health and safety of pupils have always come first. However, the existing school bus management approach still depends heavily on manual methods, including handwritten attendance registers and visual inspection, which can be ineffective and vulnerable to human error. School buses are an easy way for parents to get their kids to school in many different countries. School buses are used everywhere in the world to provide every student with a safe and secure commute from home to school. Every child on the school bus is guaranteed a safe ride, according to the technology. To ensure that every student is accounted for and gets to school and home safely every day, solutions like smart school bus systems are being implemented. (*Smart School Bus System | FMS Tech - Fleet Management System*, n.d.)

To solve these problems, the Smart School Bus System with Automated Attendance and Environment Monitoring was created by combining IoT technologies into one automated solution that guarantees safety, efficiency, and health monitoring within school buses. The system combines an ESP32 microcontroller with several sensors: MQ2 for gas monitoring, DHT11 for temperature and humidity sensing, and ultrasonic sensors for proximity sensing. From their mobile devices, teachers and parents may monitor real-time conditions, including temperature, humidity, and gas concentration levels. These sensors gather environmental data within the bus and broadcast it to a web dashboard.

1.2 Problem Statement

Although daily travel depends on school buses, conventional systems lack real-time monitoring and automation. Particularly when dealing with big groups of students, the traditional way of manually taking attendance using paper lists is sluggish, inaccurate, and ineffective. Another problem is when a student is accidentally left unaccounted for, misrecording, or omission of their name can cause uncertainty and safety concerns(Qu et al., 2022) Another important problem is the state of the bus's internal environment. Especially on lengthy journeys, enclosed bus interiors can soon become unpleasant and dangerous in the hot and humid climate of Malaysia. Among students, exposure to bad air quality, extreme heat, or trapped vehicle emissions can cause respiratory issues, tiredness, and even long-term health effects(Li et al., 2009) Buses without monitoring devices or ventilation may gather dangerous vapours like carbon monoxide or smoke, which can be damaging to the health of students.

1.3 Project Objectives

Designed to improve both safety and efficiency in school transportation, the Smart School Bus System uses embedded systems and IoT to address practical issues confronted by school administrators, parents, and students. The main goals of the project are to:

- a) To develop an automated student attendance system for a smart school bus using secure RFID-based authentication for identity verification.
- b) To design and implement a web-based dashboard for real-time monitoring of student attendance and system status, accessible by parents and school administrators.
- c) To develop a real-time IoT monitoring system using ThingSpeak for continuous data transmission of temperature, humidity, and gas level readings.

1.4 Project Scope

The project began with the development of a virtual circuit diagram to outline the system's hardware architecture using the Wokwi simulation platform. This stage helped validate the logic of the system before moving on to physical implementation. The next scope will be the work that involves establishing the physical hardware connections for all components, ensuring stable wiring and accurate communication between the sensors, microcontroller, and output devices. Along with the hardware configuration, a web dashboard interface will be created to enable real-time monitoring. The hardware system and the web dashboard will be connected in the next phase to allow for smooth data transfer, remote access, and device control. This integration will finish the transformation from a virtual simulation into a completely working IoT prototype. The project scope covers design, simulation, hardware implementation, cloud interface development, and final system integration.

1.5 Significance of the project

This project benefits school transportation management by improving how student attendance and safety inside school buses are handled. Particularly by lowering errors and delays brought on by manual attendance monitoring, it helps primary and secondary school communities, including school administration and parents. Consequently, it offers students a safer, more effective, well-managed school bus setting for their daily commute.

1.6 Expected Outcome

The expected outcome of this project is a completely operational prototype integrating automatic attendance and environmental monitoring capabilities. Utilizing secure RFID-based entry verification, the system is able to accurately record student attendance while preventing unauthorized access and duplicate entries. Real-time tracking of gas concentration levels, temperature, and humidity is given on the created online dashboard. Using the ThingSpeak cloud platform—the primary data gathering and storage medium for the system—all environmental and attendance statistics are sent and kept. Also included Telegram notifications will give parents, teachers, and

administrators immediate notifications and attendance updates so they may effectively monitor student transportation.

1.7 Project Limitation

Though the initiative meets its major goals, some restrictions are present. Instead of a genuine moving bus, the system is created as a prototype and tested in a regulated setting. It so omits motion, vibrations, or outside signal interference. Dashboard customization and data transmission frequency may also be restricted by the free version of ThingSpeak. However, features such as GPS tracking and AI-based functions were not emphasized due to cost constraints and area limitations, and they were not tested in a real-world environment.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter seeks to go through the theoretical basis and prior research pertinent to the Smart School Bus System with Automated Attendance and Environment Monitoring. Starting with the background ideas of IoT, embedded systems, and cloud platforms like ThingSpeak, it then goes into an evaluation of current systems and their drawbacks. The technological underpinnings and rationale for the design decisions for the present project are provided in this review, therefore guaranteeing its execution is based on past study and established techniques.

2.2 Review of Existing System

The key objective of this section is to examine, analyze, and compare the existing systems that have contributed to the advancement of smart attendance monitoring and environmental sensing using IoT technologies. These previous studies demonstrate how different researchers have approached the integration of embedded hardware, wireless communication, and intelligent sensing to improve safety and automation in both educational and transportation environments. However, each system presents unique strengths and limitations that have inspired the development of the proposed Smart School Bus System with Automated Attendance and Environment Monitoring using ESP32 and the web dashboard.

Five selected IoT-based systems were studied and compared with each other as well as with the proposed project, the Smart School Bus System with Automated Attendance and Environment Monitoring using ESP32. These five chosen systems are: the IoT-Based Intelligent Attendance System developed by (Nguyen et al., 2022) , Environmental Monitoring in Bus Transportation, developed by (Todorov et al., 2023) , an IoT-Based Smart School Bus Monitoring and Notification System developed by (Raj & Sankar, 2018) Secured College Bus Management System Using IoT for COVID-19 Screening, developed by (Krishnan et al., 2021), and Face Recognition Attendance System Based on Real-time Video Processing, developed by (Yang & Han, 2020).

These systems were chosen because they are relevant to the domains of environmental monitoring, smart transportation, and IoT-based automation. Every

selected study offers a different viewpoint on how IoT technology might be used to enhance school transportation systems' dependability, convenience, and safety.

2.2.1 IoT-Based Intelligent Attendance System

Project developed by (Nguyen et al., 2022) '*Internet of Things-Based Intelligent Attendance System: Framework, Practice Implementation, and Application*' incorporated face recognition technology combined with cloud-based analytics. Their system employed a Raspberry Pi 4 microcontroller with a high-definition camera, temperature sensor, and Google Firebase for real-time data storage. As pupils entered the classroom, the system instantly photographed them and compared the photographs against a pre-registered database. Once attendance was logged, the results were posted to the cloud, where teachers could view them through a web portal. Compared with manual techniques, the researchers stressed that IoT integration reduced human intervention and increased precision.

Based on various equipment needs and methods, the authors divide the attendance system into four primary categories: biometric fingerprint verification, RFID-based technology, QR code-based technology, and facial recognition. On the other hand, the writer pointed out some difficulties. The face-recognition algorithm called for good illumination and steady camera placement, which can be inconsistent in changing settings, such as moving buses. Their system also called for rather great computing capability and depended on continuous internet connectivity to sync with the cloud (Nguyen et al., 2022). These constraints highlight the need for lightweight embedded systems, such as ESP32, which can run effectively on low power.

On the other hand, the current project utilises an RFID-based identification method that reduces hardware expenses and eliminates the need for camera-based calculations. Although the Nguyen et al. method was effective for static classroom settings, the suggested Intelligent School Bus System widens the use into a mobile transportation environment where real-time response, vibration, and motion are essential. Through web dashboards rather than sophisticated cloud databases, the integration with the

ThingSpeak IoT platform also streamlines data transmission and visualization.

2.2.2 Environmental Monitoring in Bus Transportation

Environmental quality within vehicles has recently gained attention due to its direct effect on passenger health and comfort. (Todorov et al., 2023) Introduced an *Environmental Monitoring in Bus Transportation* system that used a mobile measurement platform to assess air quality parameters such as particulate matter (PM2.5), nitrogen dioxide, and ozone levels. Figure 2.2.2 explains the study involved deploying compact sensors inside city buses to collect continuous data, which were then analyzed using principal component and discriminant analysis to determine comfort indices and pollution exposure.

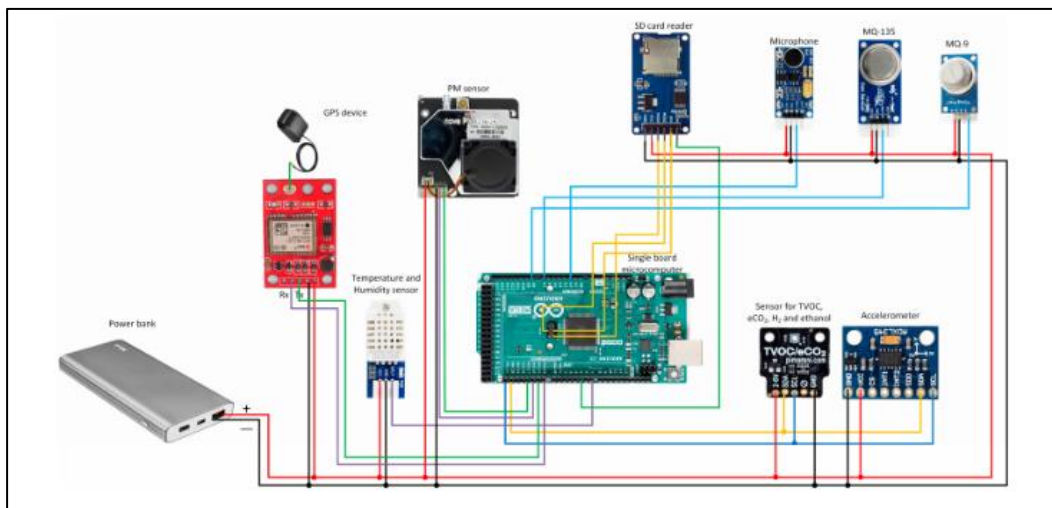


Figure 2.2.2 Simulated prototype of *Environmental Monitoring in Bus Transportation* (Todorov et al., 2023)

The experimental setup utilized in this work was constructed using the chosen hardware components. An electrical schematic of the created experimental setup is displayed in Figure 2.2.2. The Arduino Mega 2560 single-board microprocessor is connected to every device. The microphone and the sensors MQ-135 and MQ-9 are linked to the analogue inputs A7, A8, and A2. The Arduino Mega's SPI interface is linked to the SD card module, and digital pin D5 supplies the CS signal. The accelerometer and CO2 sensors are linked to the I2C communication line. Digital pin D7 is coupled

to the humidity and temperature sensor via a One-Wire interface. The hardware serial asynchronous port Serial2 is linked to the PM measurement device.

The suggested Smart School Bus System builds on this research by including a real-time approach utilizing DHT22 and MQ2 sensors wired directly to an ESP32 microcontroller. Temperature, humidity, and gas concentration (smoke or carbon monoxide) are constantly sensed by these sensors. Live uploads of the readings to the web dashboard platform enable thresholds to trigger alarms should results go over safe boundaries. Thus, while Todorov et al.'s system focused on academic data analysis, the current project transforms environmental monitoring into an immediate safety function suitable for children's transportation.

2.3.3 IoT-Based Smart School Bus Monitoring and Notification System

An IoT-Based Smart School Bus Monitoring and Notification System utilizing RFID and GPS technologies to monitor student mobility and bus location was presented by Jairam Sankar and Udy Thyparampil Raj in 2017. An ESP8266 microcontroller linked with an MFRC522 RFID reader and a GPS module created the system, which allowed for automatic student recognition during boarding and real-time transit of bus coordinates. Parents got alerts via a mobile app and cloud server when their youngster entered or exited the bus, therefore enhancing safety and communication (Raj & Sankar, 2018). Real-time tracking and automated attendance logging were effectively shown by the prototype; however, its need of Wi-Fi coverage and basic HTTP communication had drawbacks in locations with poor connectivity. RFID tag reading also necessitated proximity and regular manual scanning, hence stalling the boarding procedure. The analysts also emphasized data security as a worry because information sent on open channels could be intercepted or tampered with.

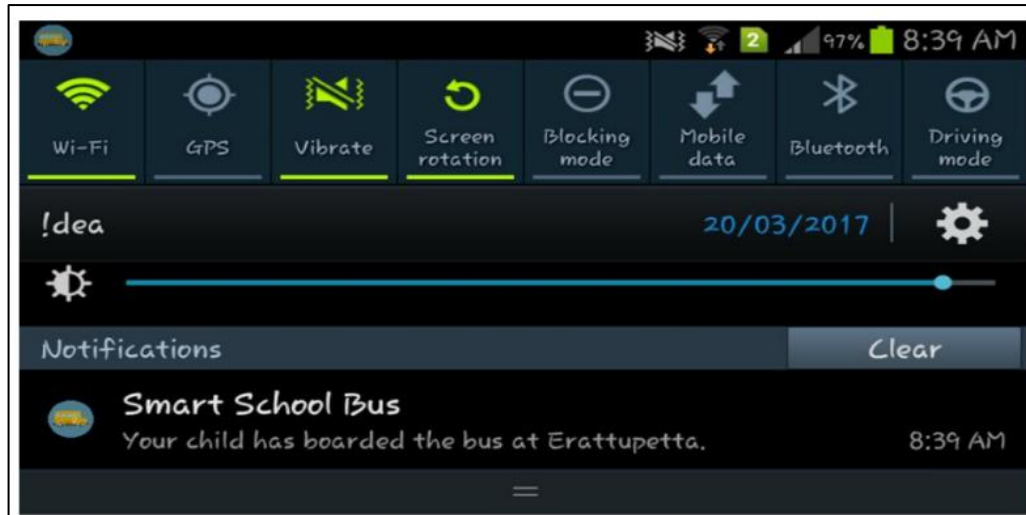


Figure 2.2.3 Interface of *IoT-Based Smart School Bus Monitoring and Notification System* (Raj & Sankar, 2018).

Figure 2.2.3 is the interface created by the developers to convey the student's location to parents and administration. To make student identification easier, a RFID-based entry system is used. Furthermore, the inclusion of environmental sensors (DHT22 and MQ2) adds extra levels of security that were lacking in Raj and Sankar's model, guaranteeing a more complete monitoring strategy.

2.2.4 Secured College Bus Management System Using IoT for COVID-19 Screening

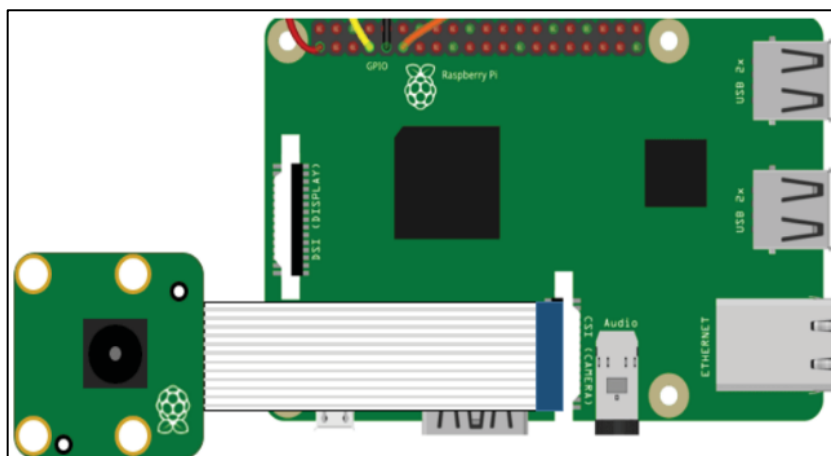


Figure 2.2.4 System setup for mask detection, *Secured College Bus Management System (SCBMS)* (Krishnan et al., 2021)

Figure 2.2.4 is the system setup for COVID-19 mask detection. This proposed system consists of 1456 datasets, of which 786 images are faces with a mask, and the remaining 670 images are faces without a mask. Once the images are trained with this dataset, it will be presented with a test data through an image or live video stream. Designed especially to improve student safety during the COVID-19 epidemic, Santhana Krishnan et al. (2021) created a *Secured College Bus Management System (SCBMS)*. The research grew from the worldwide health emergency, when safe commuting depended on social distancing, mask use, and temperature checking. To make sure only students meeting certain health standards were permitted to board the bus, the authors developed a system combining health monitoring sensors, artificial intelligence, and IoT technology. Employing a Raspberry Pi microcomputer as the main processor unit, the system included an RFID-based attendance module, an infrared temperature sensor, and a camera system trained with a MobileNet-based deep learning algorithm to automatically identify face masks.

SCBMS's operational workflow started when every student approached the bus entrance. The RFID reader first logged attendance information into the database of the system after scanning the student's ID card. The camera caught the student's facial image right after, which Mobile Net's convolutional neural network carried out mask detection; simultaneously, the IR temperature sensor measured body temperature. The pupil was allowed to board if the mask and temperature requirements were within acceptable levels. Otherwise, the system sent an automatic alert via GSM and SMS to alert parents and school officials to non-compliance or possible sickness. This multi-stage verification strategy guaranteed that every boarding event combined attendance, health screening, and traceability to offer a strong safety solution under pandemic circumstances (Krishnan et al., 2021).

The authors pointed out that their prototype successfully detected masks and validated temperature. By automating the verification procedure, the system greatly reduced physical contact between pupils and bus staff, therefore lowering transmission risk. Moreover, it gave an ordered attendance and screening results database that could help contact tracing

in case of the discovery of infection cases. Particularly helpful for remote alerting in places without internet access, GSM and SMS notifications made sure safety information got to pertinent parties on time. But the dependence of the system on Raspberry Pi hardware brought more power usage and operational cost than did microcontroller-based systems. Increased processing power and memory were also needed for camera module integration and deep-learning algorithms, thereby restricting its scalability for large fleets of school or college buses.

2.2.5 Face Recognition Attendance System Based on Real-time Video Processing

To solve problems and errors in conventional attendance tracking, Hao Yang and Xiaofeng Han (2020) developed a *Face Recognition Attendance System Based on Real-time Video Processing*. Their work used artificial intelligence and computer vision to automatically detect and log attendance in real time, therefore reducing human interference and fraud. With the OpenCV library, Python, Java, and C++ were used to implement the system, which recorded facial data via video streams, processed it for recognition, and stored attendance records in a MySQL database. It used video-based facial recognition as its main tool for identifying and authenticating student IDs. Experimental results show that when compared to manual or fingerprint-based attendance methods, the model achieved an accuracy rate of about 82%, therefore reducing class truancy by almost 60% (Yang & Han, 2020)

The scientists gave a thorough explanation of the algorithmic design, which includes Gabor wavelet transformation for enhanced resilience under lighting and expressive changes, as well as Linear Discriminant Analysis (LDA) for facial feature extraction. Creating a fully automated and continuous attendance environment, their system architecture comprised a video capture module, face recognition engine, wired transmission unit, and database management layer. Real-time video streams helped the face detection module to find and extract facial regions, which were then encoded and compared against kept facial templates for identity identification. Yang and Han showed that by adjusting to shifting

environmental circumstances and varied camera perspectives during class sessions, this continuous video-based model might outperform static image recognition solutions (Yang & Han, 2020).

On the other hand, their suggested system guaranteed non-intrusive, real-time operation while minimizing human interaction—a crucial element during pandemics like COVID-19. But the authors conceded that video blur, changes in lighting, and changes in facial features—such as hairstyles or accessories—might all impact the accuracy of the face recognition model. Particularly as the system needed several programming environments and external database administration for ongoing video streaming and recognition activities, high computational demands were also noted as a restriction.

2.3 Comparison between Similar Existing Systems and the Proposed System

Table 1: A comparison table of existing systems

Study / System	Technology Used	Limitations Identified	Relevance to Current Project
(Nguyen et al., 2022)	Raspberry Pi + Face Recognition	Designed for static classroom use; not suitable for mobile environments like buses	Shows the effectiveness of automated attendance, but lacks mobility and real-time bus safety features
(Todorov et al., 2023)	Environmental quality monitoring air quality sensors (PM, NOx, VOCs) + DHT 22	Offline system; used mainly for research; no real-time alerts	Highlights the need for real-time environmental safety monitoring
(Raj & Sankar, 2018)	IoT system with RFID + GPS + ESP8266	Lacked environmental sensors; connectivity issues in weak signal areas	Provides a foundation for bus tracking but not safety monitoring
(Krishnan et al., 2021)	COVID-19 screening (temperature + mask detection) on Raspberry Pi	High power consumption; heavy processing requirements	Useful concept of health monitoring, but too resource-intensive for bus systems
(Yang & Han, 2020)	Real-Time Video Processing, Computer Vision, (AI), Convolutional Neural Networks (CNN),	Requires strong computational resources; camera-dependent	Demonstrates accuracy, but expensive and unsuitable for moving vehicles
Proposed System (Current Project)	IoT System with ESP32 +RFID + Web Dashboard+ ThingSpeak + DHT22 (temperature and humidity) + MQ2 (detect gas)	— (addresses previous limitations)	Provides real-time environmental monitoring, low-cost attendance input, stable Wi-Fi connectivity, and web dashboard integration

Based on the comparison of existing studies, the proposed system is designed to address key limitations such as high cost, high power consumption, and lack of real-time monitoring in mobile environments. Many earlier systems depend on Raspberry Pi, AI-based processing, or industry-grade microcontrollers, which are pricey. For small-scale IoT applications, the ESP32 is chosen because it is affordable, has integrated Wi-Fi, and enough processing power, making it more workable than industry-based ARM Cortex microcontrollers. The DHT22 and MQ2 sensors are picked for their dependable temperature, humidity, and gas detection inside a tiny, confined environment like a school bus. Dashboard integration through ThingSpeak gives the suggested system a cost-effective, mobile, and real-time monitoring solution that successfully satisfies the safety and monitoring requirements discovered in earlier studies.

2.4 Limitation

Since this is my first IoT project, I am unable to implement more advanced features, such as GPS tracking or face recognition. These technologies need more funding, specialised tools, and more advanced coding ability. Lacking funds and limited knowledge, I focused on developing basic environmental sensors and simple RFID-based attendance systems. Though the system operates effectively for its intended purpose, it presently does not provide complex real-time tracking capabilities or premium, automatic identification techniques.

CHAPTER 3: METHODOLOGY

3.1 Introduction

For this project, the waterfall methodology explains each phase involved in producing the Smart School Bus System with Automated Attendance and Environment Monitoring using IoT. This chapter discusses all the steps taken to understand and fulfil the research objectives stated in Chapter 1. This chapter includes requirement analysis, system planning, circuit configuration, flowchart design, hardware and software design, system development, testing, and verification. The system is created methodically according to the selected approach, which reduces errors and boosts productivity all along the way. This methodical approach gives the project a clear direction and makes it easier to document, implement, and assess the finished prototype.

3.2 System Overview

Using a variety of diagrams, including block diagrams, circuit diagrams, flowcharts, and system images, we will go into detail on the Smart School Bus System with Automated Attendance and Environment Monitoring in this session. The primary controller for all the system's integrated sensors and actuators is the ESP32 microprocessor. The ESP32 oversees the system's three sensors and five actuators. The smart school bus system's block diagram and pictures are shown below.

3.2.1 Block Diagram

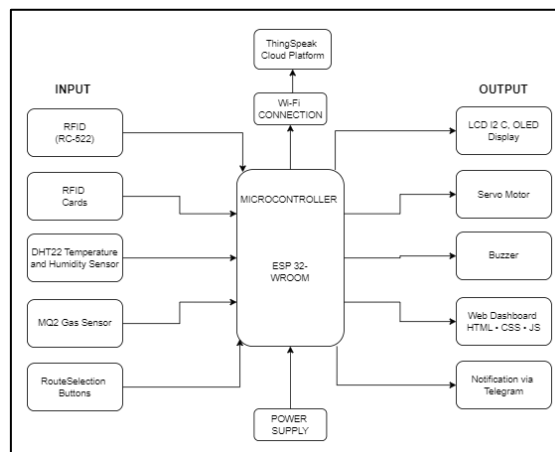


Figure 3.1 Block diagram

Figure 3.1 illustrates the overall structure of the Smart School Bus Monitoring System, which is controlled by an ESP32-WROOM microcontroller. The system uses several input sensors, such as a pushbutton, RFID tags for student input, a DHT11 sensor for temperature and humidity monitoring, and an MQ2 sensor for gas detection. The ESP32 receives real-time data from these sensors continually. Remote monitoring and real-time notifications are made possible by the ESP32's processing of all incoming signals and communication with ThigSpeak platform and the web dashboard mobile application via an internet connection. The system operates the output side with components such as a buzzer for status alerts, an LCD I2C display, an OLED display, and a servo motor for door control. Every time certain things happen, including successful student authentication or unusual gas levels, notifications are delivered to the Telegram app. A controlled power source powers the entire system, ensuring that the microcontroller, actuators, and sensors all operate steadily. The school bus's fully automated attendance verification and environmental monitoring systems are supported by this arrangement.

3.2.2 Circuit Diagram

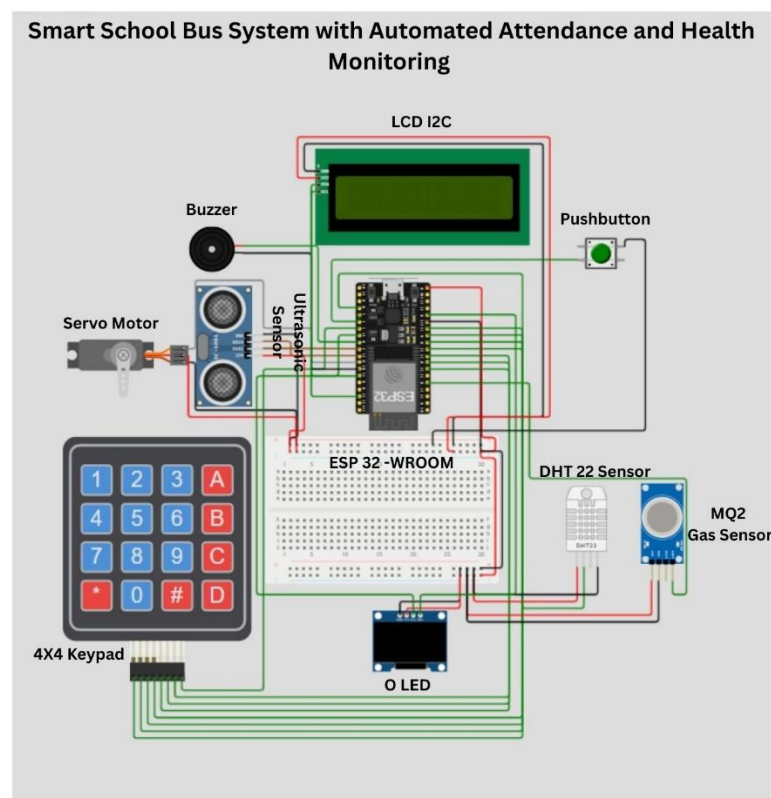


Figure 3.2 Wokwi Project Circuit Diagram

Figure 3.2 shows how all sensors, input devices, and output components are electrically connected to the ESP32-WROOM microcontroller. Each module is wired to the appropriate GPIO pins to ensure stable signal flow and proper system operation. The DHT22 and MQ2 sensors are wired to the ESP32 to measure temperature, humidity, and gas levels. The HC-SR04 ultrasonic sensor is connected to the trigger and echo pins for distance detection. A 4×4 keypad is linked through multiple GPIO pins to record student attendance inputs. For outputs, the servo motor is connected to a PWM pin to control the door mechanism, while the buzzer is wired to a digital pin for alert notifications. Two displays—the LCD I2C and OLED module—share the ESP32's I2C interface to present system information. A pushbutton is included for manual activation or reset. All components receive stable power from the 3.3V or 5V outputs of the ESP32, with a shared ground ensuring stable and safe operation. This wiring setup ensures smooth data flow and reliable operation of the smart school bus system.

For the circuit simulation, I created a Wokwi online circuit simulation for the proposed system (*Wokwi - Online ESP32, STM32, Arduino Simulator*, n.d.). I used a 4x4 keypad as a replacement device for RFID due to Wokwi's limitations.

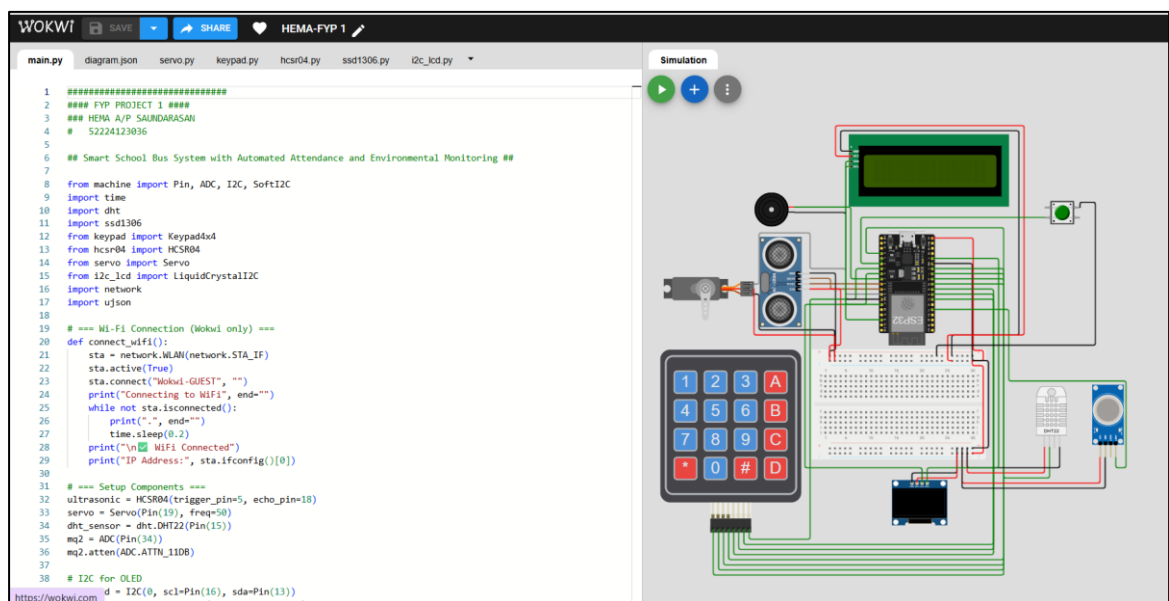


Figure 3.3 Wokwi project dashboard

3.2.3 Flowchart

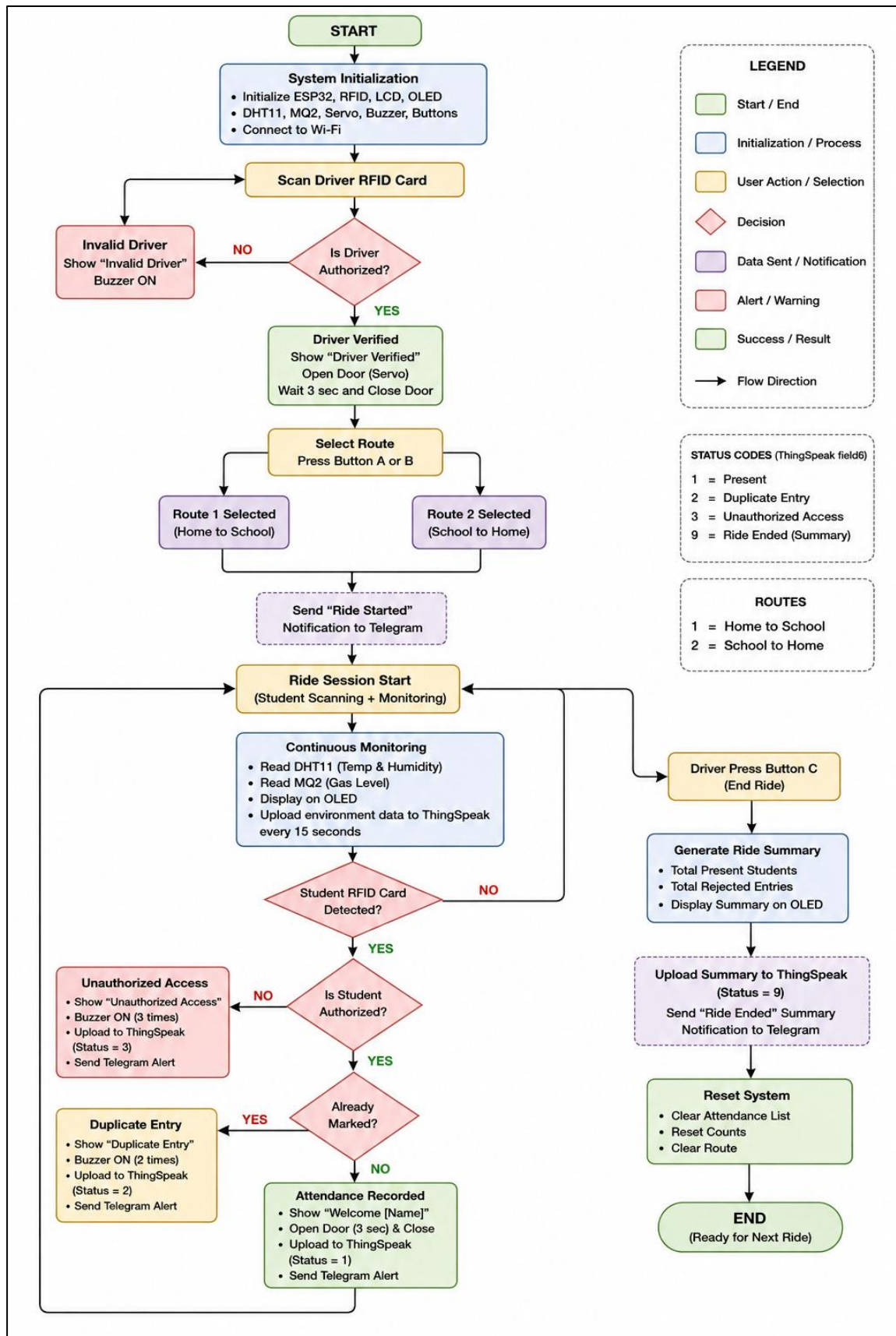


Figure 3.4 Project flowchart

Figure 3.4 is the system's flowchart, which shows the logical steps taken by the Smart School Bus System with Automated Attendance and Environment Monitoring. The ESP32 first starts its Wi-Fi connection to connect with the ThingSpeak cloud. Once the link has been created, it continually scans environmental sensor readings (gas level, humidity, and temperature). For immediate real-time display, these readings are sent to the web dashboard.

The system waits for a student to tap the RFID card that is assigned to each student for attendance. Against pre-stored values, the ESP32 validates the attendance. Attendance is noted if the entry is valid; a confirmation message shows on the LCD. Once the student enters, the servo motor opens the door, then shuts it. The buzzer is triggered to inform the user if the entry is inaccurate or repeated. This organised flow guarantees attendance is handled quickly, and environmental data is regularly updated.

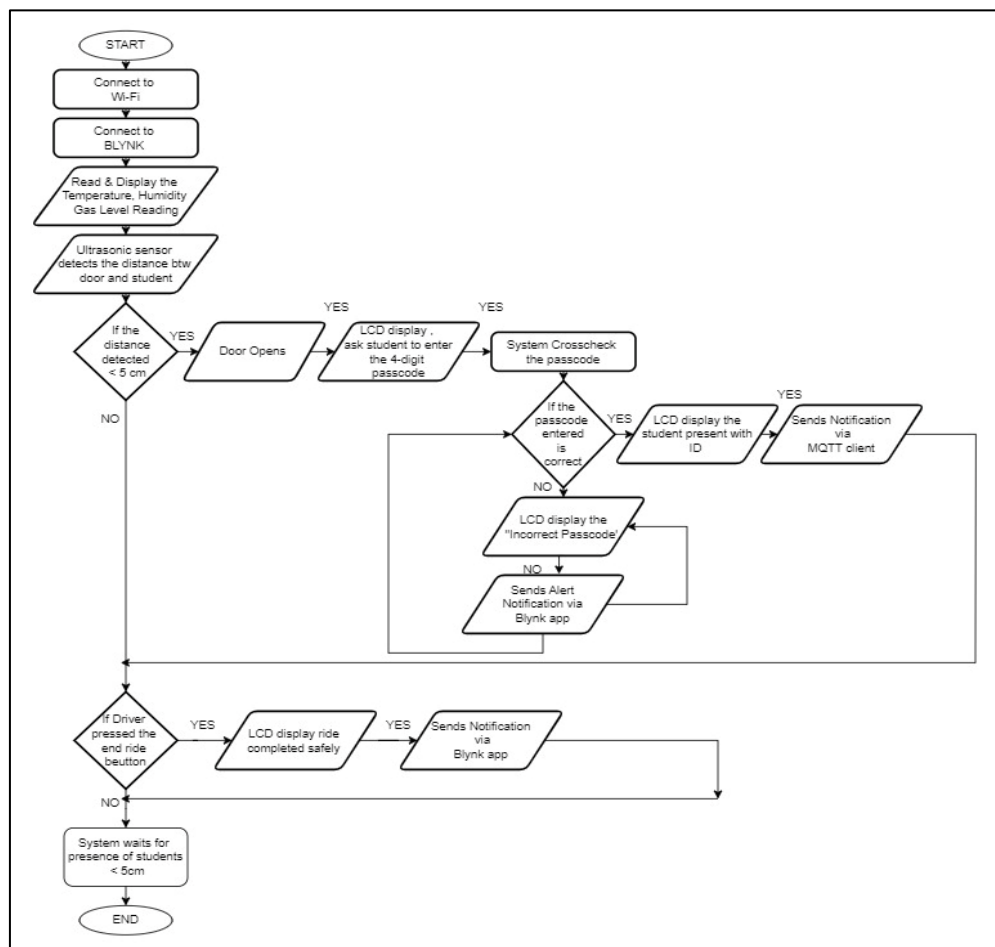


Figure 3.5 Wokwi Simulation's Flowchart

The flowchart for the Wokwi simulation is shown in Figure 3.5 In order to connect to the Blynk cloud, the ESP32 first establishes a Wi-Fi connection. After the link is established, it continuously checks the temperature, humidity, and gas level measurements from environmental sensors. These readings are transmitted to the Blynk mobile application for instantaneous real-time display.

Each student is given a four-digit passcode for attendance, which the system waits for them to type into the keypad. The ESP32 verifies the passcode against pre-stored values. Attendance is recorded upon valid passcode entry, with a confirmation message displayed on the LCD. The servo motor opens and closes the door for the student, while a buzzer sounds for inaccurate or repeated entries, ensuring efficient attendance management and regular environmental data updates.

3.2.4 Control modes of the system

In this project, the Smart School Bus System with Automated Attendance and Environment Monitoring relies on two centralised control modes: the Web Dashboard, ThingSpeak cloud platform, and Telegram push notification. Data visualisation, alert management, and real-time monitoring are primarily handled by this platform.

a. Web Dashboard

As the main user interface for the entire system, the web dashboard is essential. ThingSpeak is a low-code platform that integrates secure IoT cloud hosting, device connectivity, and customisable apps. For hardware teams, it's the quickest approach to develop, implement, and grow linked products. It converts real-time sensor data into visual dashboards so that users may quickly comprehend the internal environment of the bus and the state of the students. All updates, including environmental readings, attendance logs, and door status, are promptly reflected on the application thanks to the ESP32's Wi-Fi communication with the ThingSpeak cloud.

3.2.5 Push Notifications

a. Telegram Notifications

In order to guarantee that users receive critical alerts promptly and consistently, Telegram notifications are incorporated as an extra alert method. Telegram's multi-device availability—which allows messages to show immediately on smartphones, tablets, and computers—enables administrators to receive notifications using the Telegram application (*Telegram Bot API*, n.d.). Moreover, because it offers a free messaging service, Telegram is cheaper than SMS-based systems that charge for each message. Employing Telegram provides a reliable, reasonably priced, and fast secondary alerting channel that improves safety by means of ongoing and rapid contact.

3.3 The Methodology Model

One of the most widely used methods in project development is the waterfall methodology. This life cycle model is regarded by most companies as a traditional approach to software development. The project development process is explained in a linear sequential flow using this paradigm. You should always double-check if the previous step is finished at every stage of the development cycle. This conventional approach to development is based on a strict linear model. This method of development does not specify how to address requirements modifications by going back to the earlier stage. (*Top 12 Software Development Methodologies - TatvaSoft Blog*, n.d.)

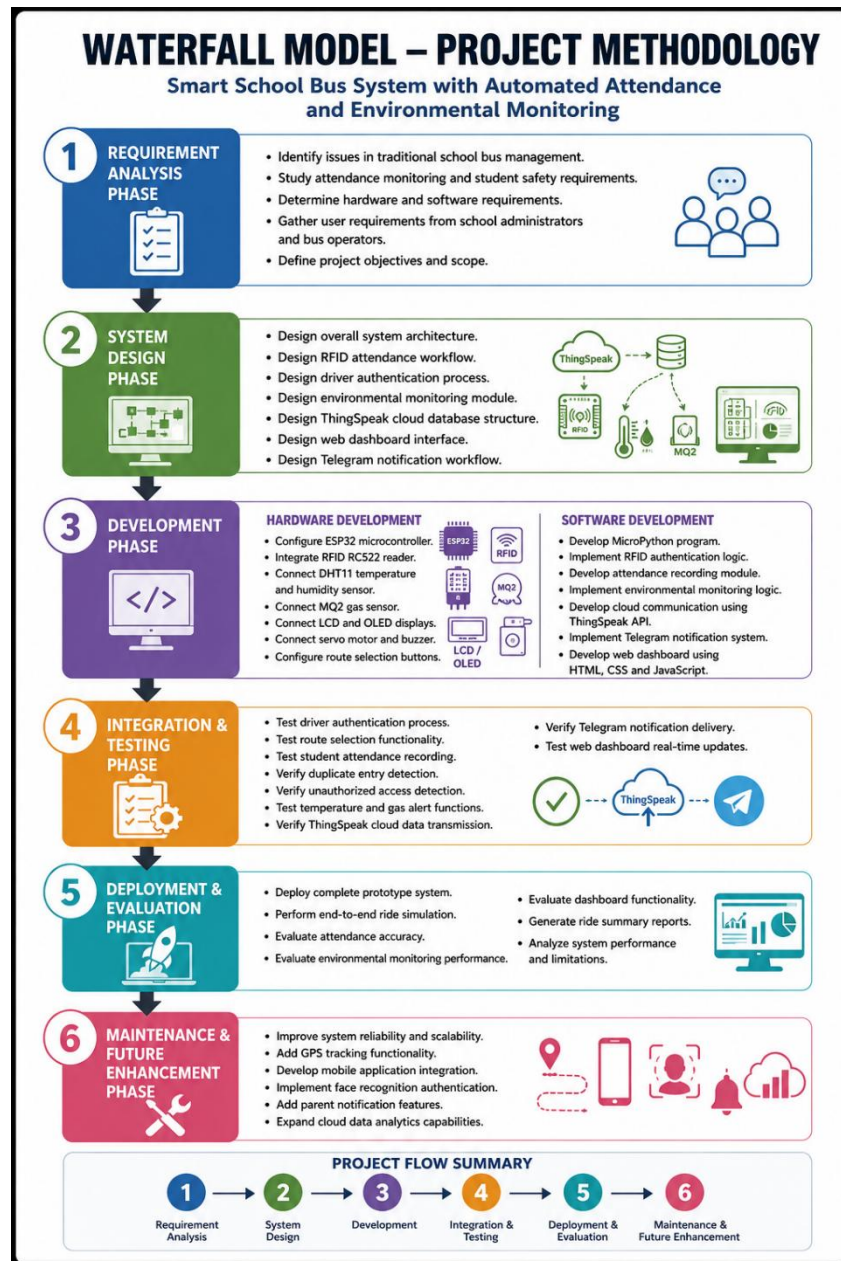


Figure 3.6 Waterfall model methodology (*Images (300×168), n.d.*)

3.31 Requirement Phase

The Requirement Phase is the foundation of the Waterfall methodology and focuses on identifying all the essential needs of the proposed system. During this phase, I looked for a project, then started to do research after the title was authorized, by studying related papers and current journals to grasp the current technology and weaknesses in traditional school bus systems. While conducting this research, I was able to identify the limitations of existing projects, such as a lack of real-time

monitoring, limited system integration, and high implementation costs. Furthermore, through the literature review, I identified the appropriate hardware components required for the system, including the ESP32 microcontroller, ultrasonic sensor, DHT sensor, and other supporting modules to make this project happen. I have also got the idea of including creating a web dashboard in this system to make it user-friendly.

Furthermore, in this stage, I obtained all the required data on the sensors and parts required for my system. This helps one understand their roles, capabilities, and how they support automated attendance, environmental monitoring, and safety systems.

3.32 Design Phase

The needs gathered previously are translated during the Design Phase into a thorough technical plan for the system. This process entails producing the system architecture via a few vital design sketches. As illustrated in Figure 3.1, the block diagram offers a high-level visual summary of the interactions among the major system elements. It clarifies the general flow of operations and the link between each functional element of the system. The circuit diagram in Figure 3.2 gives a more exact picture by showing that every component is connected. The step-by-step logic and processes engaged in the system are shown in the flowchart in Figure 3.4. It clarifies the order of operations, decision-making sites, and the system's reaction to many circumstances. Moreover, this is the phase where I design the RFID attendance workflow, driver authentication process, ThingSpeak cloud database structure, web dashboard interface and Telegram notification workflow.

At this phase, all design work is still paper-based and visual; these created schematics help to vividly picture the project. The hardware arrangement and physical development will go on in my Implementation phase.

3.3.3 Implementation Phase

The Implementation Phase focuses on converting the design into a working system. Starting with the hardware connections, the circuit design

links the ESP32 microcontroller to all sensors and output devices. Every part is power tested to guarantee consistent performance. Software development is done using the Thonny IDE once the hardware setup is finished; here, the ESP32 is coded to read sensor data, check entry input, drive the servo motor, and communicate with ThingSpeak platform. Developing a web dashboard using HTML, CSS and JavaScript also plays the main role of the system. This step also entails using mobile dashboard displays of temperature, humidity, gas levels, attendance records, and door status via web dashboard interface widgets. Real-time notifications for emergency events, including excessive gas levels or unusual temperature readings, are made possible by the Telegram Bot API setup. To guarantee quick system reactions, event triggers and automation algorithms are used. Finally, all modules—attendance, environmental monitoring, and notifications—are combined into a single integrated system and tested for synchronized operation.

3.3.4 Testing Phase

The testing Phase guarantees that the system meets the specifications set out in the earlier steps. In this stage, various testing methods are considered to ensure that every part of the Smart School Bus System functions accurately and reliably.

Integration testing is a primary technique in which several system modules are tested together. For instance, testing simultaneously would be the interaction between the RFID module and the attendance system, the communication between sensors and the ESP32, and the connection from the ESP32 to platforms like ThingSpeak and Telegram. This guarantees that parts that operate independently can also run smoothly when merged, without generating conflicts or data flow problems.

Functional testing—that assesses if every component of the system operates per the specified needs—is another significant approach. Functional testing for this project would verify whether the attendance verification process operates properly, if environmental readings trigger alerts as expected, and whether notifications are sent correctly when

specific events occur. This confirms that every system component works exactly as designed.

Before proceeding to the next stage, these two testing methods are sufficient at this point to ensure that important system features and interactions are verified.

3.3.5 Deployment and Evaluation Phase

During this stage, a finished Smart School Bus Monitoring and Attendance System prototype was implemented and tested under simulated operating conditions. Starting with driver authentication and path selection to student attendance tracking, environmental monitoring, and ride summary creation, a comprehensive ride simulation was carried out. The RFID attendance module was examined to guarantee correct student identification free from unapproved and duplicate admissions. The DHT11 and MQ2 sensors were tested to confirm correct gas level, temperature, and humidity monitoring. Also evaluated were the web dashboard and ThingSpeak cloud platform to guarantee real-time data transmission and display success. Finally, the general system performance, features, and constraints were examined to evaluate how well the suggested solution worked and pinpoint areas for future development.

3.3.6 Maintenance Phase

The final phase, Maintenance, focuses on ensuring the system's long-term stability and performance after deployment. The end users, the manufacturing business, and I, as the developer, define the maintenance responsibilities in this project.

For the hardware components, the maintenance largely depends on the manufacturing companies because I am also a secondary user of the sensors and modules used in the system. Any defects, lifespan limitations, or calibration issues related to components such as the RFID, microcontroller, or other modules would follow the manufacturer's warranty and support policies. For the software and coding, I plan to provide a warranty period of 6 months to 1 year. During this time, I will address minor

system bugs, logic adjustments, and improvements that may be needed based on user experience.

The maintenance phase guarantees that both hardware and software remain dependable, with responsibilities distributed accordingly among the users, the vendors, and me as the system developer.

3.4 Cost of the Project

The table 2 ; project cost describes the approximate amount of money needed to create the Smart School Bus System with Automated Attendance and Environment Monitoring. It contains the specifications and individual costs of every piece of hardware utilized in the prototype. Compatibility, dependability, and appropriateness for automation and real-time monitoring have all been taken into consideration while choosing each component. To ensure transparency in resource allocation and budgeting, each item is displayed for clarity based on its nature and role within the system.

Table 2: Cost of the project

No.	Component	Quantity	Unit Price (RM)	Total Price (RM)
1	ESP32 WROOM-32 Microcontroller	1	35.00	35.00
2	PushButtons	3	2.50	7.50
3	DHT11 Temperature & Humidity Sensor	1	15.00	15.00
4	MQ2 Gas Sensor	1	10.00	10.00
5	RFID Card & Card Reader	1	18.00	18.00
6	Servo Motor	1	12.00	12.00
7	OLED Display	1	15.00	15.00
8	LCD Display	1	18.00	18.00
9	Buzzer	1	3.00	3.00
10	Jumper Wires & Breadboard	3 set	10.00	30.00
11	Power Supply / Adapter (5V–12V)	1	15.00	15.00
12	Miscellaneous (Prototype materials)	1	70.00	70.00
	TOTAL		RM 248.00	

3.5 System Requirement Analysis

3.5.1 Hardware Requirement

Table 3: List of hardware components and purposes

Hardware Component	Purpose
ESP32 Microcontroller	Controls system operations and Wi-Fi communication.
RFID RC522 Reader	Reads RFID cards for driver and student authentication.
RFID Cards/Tags	Store unique user identification information.
DHT11 Sensor	Measures temperature and humidity levels.
MQ2 Gas Sensor	Detects gas and smoke concentration.
16×2 LCD Display	Displays attendance and system status messages.
OLED Display	Displays real-time environmental readings.
Servo Motor	Simulates automatic bus door operation.
Buzzer	Generates warning and alert sounds.
Push Buttons	Used for route selection and ride termination.
Breadboard & Jumper Wires	Provide circuit connections between components.
Power Supply	Supplies power to the hardware components.

The physical framework of the Smart School Bus System as stated in Table 3 utilizes a modular hardware architecture where specialized peripheral modules connect directly to a central computing core. At the center of the system, the ESP32 Microcontroller serves as the primary processing unit, executing embedded logic routines and uploading real-time data to the cloud via its built-in Wi-Fi module. Student and operator authentication is handled by the RFID RC522 Reader and Cards/Tags, which automates the identification process by scanning unique ID strings to log passenger attendance instantly without manual ledger entries. Environmental cabin safety is continuously checked using a dual-sensor array consisting of the DHT11 Sensor to track ambient cabin temperature and humidity levels, alongside the MQ2 Gas Sensor to identify toxic exhaust gases or sudden

smoke leaks. Visual status feedback is split across a dual-display setup where a 16×2 I2C LCD Display outputs administrative text, such as passenger counts and system status messages, while an independent OLED Display presents live microclimate sensor metrics. Physical and auditory system responses are executed through a Servo Motor, which rotates to simulate an automatic entry door opening upon successful authentication, and a Buzzer that triggers loud warning frequencies during environmental hazard spikes or failed card validations. Finally, the foundational system infrastructure is maintained by Push Buttons that allow the driver to configure route settings and end transit sessions, all routed and powered through an interconnected matrix of a standard Breadboard, Jumper Wires, and a stable Power Supply module.

3.5.2 Software Requirement

Table 4: List of Software components and purposes

Software	Purpose
MicroPython	Programming language used for ESP32 development.
Thonny IDE	Used to write, upload, and debug MicroPython code.
ThingSpeak	Cloud platform for storing and visualizing IoT data.
Telegram Bot API	Sends real-time attendance and alert notifications.
HTML	Develops the structure of the web dashboard.
CSS	Designs the user interface of the web dashboard.
JavaScript	Enables dynamic dashboard functionality and data retrieval.
Wi-Fi Network	Provides internet connectivity for cloud communication.
Web Browser	Accesses and monitors the dashboard interface.

To guarantee ongoing transit monitoring, the Smart School Bus System's software environment handles web visualisations, cloud processing, and embedded execution as stated in Table 4. Using MicroPython in the Thonny IDE—which helps to write, compile, and upload script logic to the main ESP32 chip—embedded firmware development is addressed. While the Telegram Bot API serves as the communication layer to send automatic push alarms straight

to stakeholder mobile devices, remote data management depends on the ThingSpeak cloud platform to store incoming telemetry and produce historical data charts. Using a regular web stack where HTML creates the interface layout, CSS styles the responsive visual theme, and JavaScript actively pulls raw data lines from the cloud database via asynchronous API calls, the user-facing monitoring portal is built. At last, a local Wi-Fi Network manages the physical-to-cloud data bridge from the moving bus chassis, therefore letting users safely see live operations via any ordinary Web Browser.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the implementation, testing, results, and discussion of the Smart School Bus Monitoring and Attendance System. The system was developed using an ESP32 microcontroller integrated with RFID technology, environmental monitoring sensors, cloud services, and a web-based dashboard. The project transitions from raw concepts into a working hardware prototype run by a custom MicroPython script. The implementation phase involved both hardware and software integration to ensure reliable attendance and environmental condition monitoring, as well as real-time communication with cloud platforms.

The developed system was evaluated through a series of functional tests to verify its performance and reliability. The testing process focused on driver authentication, student attendance recording, environmental monitoring, cloud data transmission, dashboard visualization, and Telegram notification services. The results of these tests are discussed to determine whether the project's objectives were achieved.

4.2 Hardware Implementation

In order to create a working Smart School Bus Monitoring and Attendance System, different electronic components have to be integrated during the hardware implementation phase. The hardware architecture was

created to provide connectivity with cloud-based platforms, environmental monitoring, safety alarm creation, driver authentication, and student attendance tracking. The core processing unit that oversaw all hardware functions and communication procedures was the ESP32 microcontroller.

Component selection, circuit development, sensor integration, output device configuration, and prototype assembly were all part of the hardware implementation process. To guarantee dependable performance, each part was verified separately before being incorporated into the overall system.

4.2.1 Microcontroller Configuration

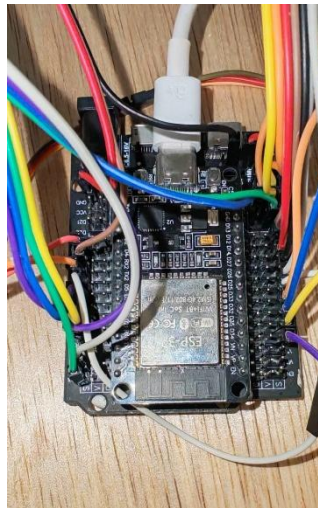


Figure 4.1 ESP32 Development Board

As shown in Figure 4.1, the ESP32 microcontroller was selected as the main controller for the proposed system. Engineered for mobile devices, wearable electronics and IoT applications, ESP32 achieves ultra-low power consumption with a combination of several types of proprietary software.

ESP32 also includes state-of-the-art features, such as fine-grained clock gating, various power modes and dynamic power scaling. With built-in antenna switches, RF baluns, power amplifiers, low-noise receive amplifiers, filters, and power management modules, the ESP32 is highly integrated. Additionally, it can function as a slave device to a host MCU or as a fully independent system, which lowers the communication stack overhead on the primary application CPU. With

its SPI/SDIO or I2C/UART interfaces, the ESP32 can interface with other systems to offer Bluetooth and Wi-Fi capabilities.(*ESP32 Wi-Fi & Bluetooth SoC | Espressif Systems*, n.d.)

Using the Thonny Integrated Development Environment (IDE), MicroPython was used to program the microcontroller. The ESP32 sets up a wireless connection to the configured Wi-Fi network before turning on all linked devices. The system goes into operating mode once the initialization procedure is finished and starts checking the surroundings and attendance.

Using the Serial Peripheral Interface (SPI) protocol, the ESP32 interacts with the RFID reader and OLED display. The Inter-Integrated Circuit (I2C) protocol helps to reduce the number of pins needed for the LCD display. The MQ2 gas sensor offers analogue output values processed by the ESP32's Analog-to-Digital Converter (ADC); the DHT11 sensor interacts via a digital interface. The servo motor is also controlled with Pulse Width Modulation (PWM) signals produced by the microcontrollers.

The ESP32's built-in Wi-Fi feature helps with Telegram notification APIs and interaction with ThingSpeak cloud services. This capability lets the system do remote notifications and real-time monitoring free from the need for more communication modules.

4.2.2 Input Devices

Input devices are any peripherals used to provide data and control signals to a computer, allowing the user to put data into the computer. Without any input devices, the system would only be a display device and not allow users to interact with it.(*CSCA0101 Computing Basics Chapter 3 Input Devices*, n.d.)

Table 5: Input hardware devices and the function

Component	Function
RFID RC522 Reader	Student and driver identification
DHT11 Sensor	Temperature and humidity monitoring
MQ2 Sensor	Gas and smoke detection
Push Buttons	Route selection and ride control

Table 5 explains the hardware components used in the proposed Smart Bus Monitoring and Attendance System and their respective functions. Each component plays a specific role in enabling student identification, environmental monitoring, route selection, and overall system operation.

a) RFID RC522 Reader



Figure 4.2 MFRC522 RFID Reader Module

The RFID RC522 reader in Figure 4.2 plays an important role in the system by identifying authorized users, such as students and drivers, through unique RFID cards. It compares the card's unique identification number (UID) with a database in the microcontroller. Valid driver cards grant access to transportation sessions, while student cards automatically record attendance and upload data to the cloud. Unauthorized cards result in alerts being generated.

b) DHT 11 Temperature & Humidity Sensor

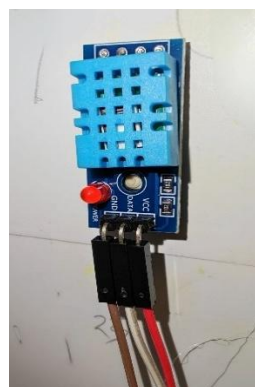


Figure 4.3 DHT11 Temperature and Humidity Sensor

The DHT11 sensor in Figure 4.3 monitors temperature and humidity inside school buses, ensuring passenger comfort and safety. It continuously measures ambient conditions, transmits data to the ESP32 microcontroller, displays readings on an OLED screen, and uploads information to ThingSpeak for remote analysis.

c) MQ2 gas sensor



Figure 4.4 MQ2 Gas Sensor

The MQ2 gas sensor in Figure 4.4 detects smoke and hazardous gases in buses, providing analogue values related to gas concentration. When readings surpass safety thresholds, it activates alarms and visual alerts, thereby enhancing passenger safety through early detection of dangerous conditions.

d) Pushbuttons

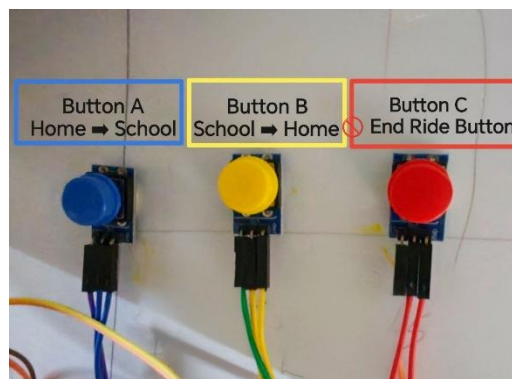


Figure 4.5 Push Buttons for Route Selection

Three push buttons are included in the system: two for selecting transportation routes ("Home to School" and "School to Home") and

one for terminating the ride session and displaying the summary. Route selection occurs before attendance monitoring starts.

4.2.3 Output Devices

Output devices are hardware components that show or generate the outcomes of a computer's processing. They transform digital data into representations that are visible, audible, or tactile. (*Output Devices - GeeksforGeeks*, n.d.)

Table 6: Output hardware devices and their function

Component	Function
LCD Display	Attendance and system status display
OLED Display	Environmental monitoring display
Servo Motor	Automatic door simulation
Buzzer	Audio alert generation

Table 6 explains the output hardware devices used in the Smart Bus Monitoring and Attendance System and their respective functions. These components provide visual and audio feedback, display system information, and simulate automatic door operation.

a) I2c LCD



Figure 4.6 16×2 LCD Display

A 16×2 LCD module in Figure 4.6 was utilized to present attendance information and system messages, offering real-time feedback to students and drivers during authentication and attendance recording. The messages include driver verification status, route selection

confirmation, attendance confirmation, duplicate entry warnings, and unauthorized access notifications

b) OLED



Figure 4.7 OLED Display Module

The OLED display in Figure 4.7 serves as the main interface for environmental monitoring, continuously showing real-time readings of temperature, humidity, and gas levels from the DHT11 and MQ2 sensors, allowing users to observe current conditions directly.

c) Servo Motor



Figure 4.8 Servo Motor

The servo motor in Figure 4.8 was integrated to simulate the automatic operation of a school bus door, opening upon successful user authentication and returning to close it. This implementation illustrates the integration of physical access control mechanisms in transportation applications.

d) Buzzer

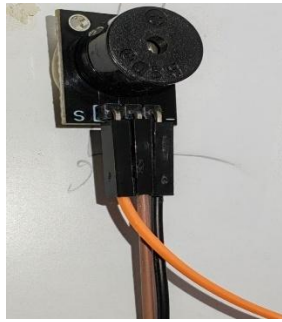


Figure 4.9 Buzzer Module

The buzzer in Figure 4.9 functions as an audible alert device. It generates warning sounds whenever abnormal situations occur, including unauthorized access attempts, duplicate attendance entries, excessive temperature conditions, and dangerous gas concentrations.

4.2.3 Circuit Design and Connection

Table 7 and Figure 4.11 below explain the wiring connection of the hardware circuits. The hardware parts were linked using a mix of I2C, SPI, PWM, digital input, and analogue input communication modes. The circuit was painstakingly created to guarantee consistent communication between the ESP32 and all linked peripherals. The RFID RC522 reader was linked to the ESP32 via committed Serial Clock (SCK), Master Output Slave Input (MOSI), Master Input Slave Output (MISO), and Chip Select (CS) GPIO pins employing SPI communication. This setup lets card validation methods communicate at great speed.

The OLED display was set up using SPI communication as well because it allows quick graphical changes. Using an I2C interface helped the LCD module to save GPIO resources and simplify wiring complexity.

For temperature and humidity measurement, the DHT11 sensor was linked to a digital GPIO pin; for analog gas concentration measurement, the MQ2 gas sensor was connected to an ADC-capable pin. The PWM-capable pin connected the servo motor so that exact rotational control would be possible.

The buzzer and pushbuttons were wired to digital input/output pins to enable user contact and alert generation. To guarantee consistent system operation and reduce signal interference, correct power distribution and grounding systems were used.

Table 7: Hardware devices connection table

Component	ESP32 Pin(s)	Connection Type
RFID RC522 Reader	GPIO 18 (SCK), GPIO 23 (MOSI), GPIO 19 (MISO), GPIO 5 (SS/SDA)	SPI
LCD 16×2 Display (I2C)	GPIO 21 (SDA), GPIO 22 (SCL)	I2C
OLED Display	GPIO 14 (SCK), GPIO 13 (MOSI), GPIO 17 (DC), GPIO 27 (RES), GPIO 26 (CS)	SPI
DHT11 Temperature and Humidity Sensor	GPIO 25	Digital Input
MQ2 Gas Sensor	GPIO 34	Analog Input (ADC)
Buzzer	GPIO 16	Digital Output
Servo Motor	GPIO 33	PWM Output
Push Button A	GPIO 32	Digital Input
Push Button B	GPIO 35	Digital Input
Push Button C	GPIO 2	Digital Input
Wi-Fi Module (Built-in ESP32)	Internal ESP32 Module	Wireless Communication

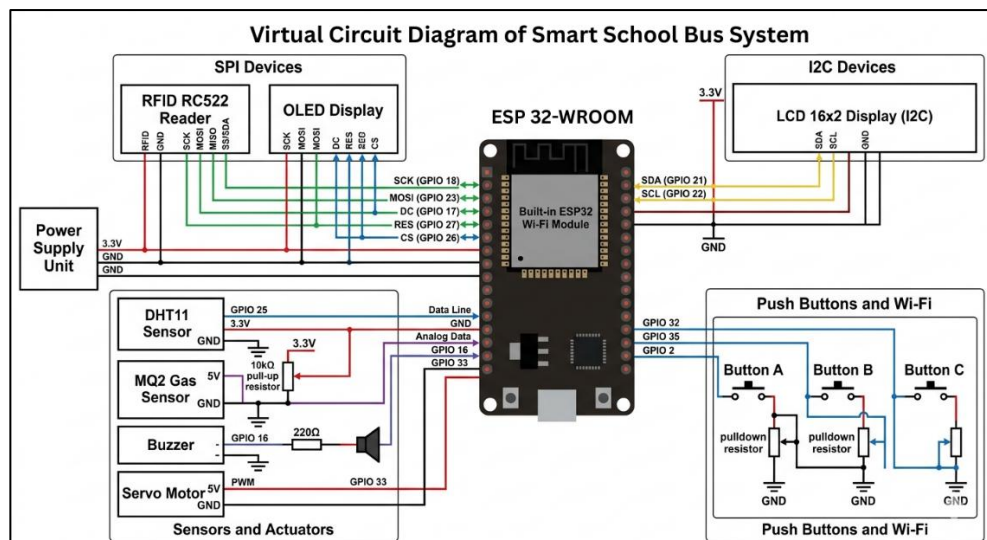


Figure 4.10 Complete Virtual Circuit Diagram

4.2.4 Prototype Development

Following successful circuit implementation, all hardware components were assembled into a functional prototype representing the proposed Smart School Bus Monitoring and Attendance System, as shown in Figures 4.11, 4.12, 4.13 and 4.14. The prototype was developed to demonstrate the practical implementation of the designed architecture and validate system functionality under real operating conditions.

The prototype integrates attendance monitoring, environmental sensing, cloud communication, notification services, and safety monitoring into a single platform. The ESP32 continuously coordinates interactions between sensors, displays, communication modules, and cloud services throughout system operation.

During testing, the prototype successfully authenticated drivers, recorded student attendance, monitored environmental conditions, generated alerts, transmitted data to ThingSpeak, and delivered Telegram notifications. These functionalities demonstrate the feasibility of implementing the proposed system in real school transportation environments.

The completed prototype serves as proof of concept for an intelligent transportation monitoring solution capable of enhancing student safety, attendance management, and operational efficiency.



Figure 4.11 Right Side view of the prototype created



Figure 4.12 Front view of the prototype created



Figure 4.13 Left Side view of the prototype created

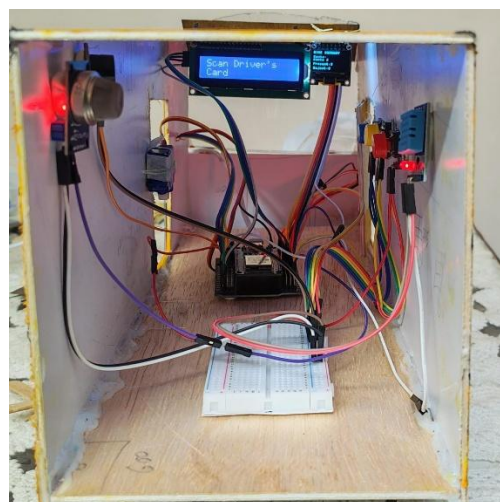


Figure 4.14 Back view of the prototype

4.3 Software Development

Designing and putting the control logic needed for the Smart School Bus Monitoring and Attendance System was the subject of the software development phase. Using the MicroPython programming language, the software was created and run on the ESP32 microcontroller through Thonny Integrated Development Environment (IDE).

The produced program combines attendance tracking, environmental monitoring, cloud communication, dashboard display, and notification features into one platform. While regulating output devices and interacting with outside cloud platforms, the ESP32 constantly analyzes input data gathered from sensors and RFID equipment. The software architecture was created with a modular style whereby every component serves a certain purpose. This design makes maintainability, clarity, and system reliability better.

4.3.1 System Flow Implementation

The project starts with system initialization and concludes with ride completion. The system operates in an organized manner as shown in Figure 3.4. The program continuously keeps an eye on system inputs and, based on the data it receives, carries out predetermined actions.

The ESP32 connects to the configured wireless network when the system is turned on. All hardware peripherals, including the RFID reader, displays, sensors, servo motor, and buzzer, are initialized upon a successful Wi-Fi connection.

Before any transportation operations may start, the driver must authenticate using a valid RFID card. The driver uses the available push buttons to choose the transit route after verification is successful. Home to School and School to Home are among the routes that are accessible.

Students scan their RFID cards to log their attendance throughout the ride session. Every RFID card is checked by the system against the approved student database. Unauthorized cards and duplicate scans

result in alerts and warnings, although valid students are noted as present.

Concurrently, data on temperature, humidity, and gas concentration are continuously gathered by the environmental monitoring subsystem. The gathered data is periodically transferred to the ThingSpeak cloud platform and shown on the OLED panel. The driver uses the assigned push button to end the riding session when it is finished. After that, a ride summary is created and sent to the Telegram notification service and cloud platform.

4.3.2 API Design and Development

Application Programming Interfaces (APIs) were implemented to enable communication between the ESP32 microcontroller and external services. The developed APIs provide real-time communication with cloud storage and notification platforms.

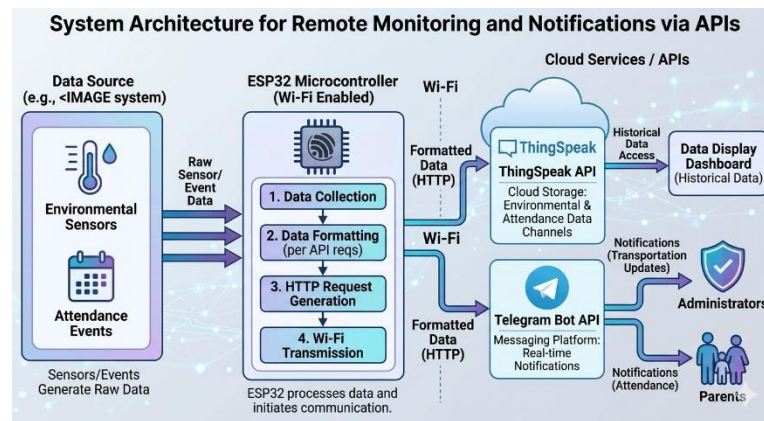


Figure 4.15 Data Flow Architecture (ESP32 to Cloud APIs)

Two APIs were integrated into the system:

- ThingSpeak API
- Telegram Bot API

The ThingSpeak API is responsible for storing environmental and attendance data in cloud channels. The Telegram Bot API is used to transmit attendance notifications and transportation updates to administrators and parents.

The API integration process allows the developed system to extend beyond local hardware monitoring and provide remote access to transportation information.

The communication process begins when data is generated by sensors or attendance events. The ESP32 formats the data according to the API requirements before transmitting it through HTTP requests over Wi-Fi. The successful implementation of API communication enables real-time monitoring and improves overall system accessibility.

4.3.3 ThingSpeak Cloud Platform Integration

Real-time data logging, visualisation, and remote monitoring were made possible by integrating the ThingSpeak cloud platform into the Smart School Bus Monitoring and Attendance System. From the ESP32 microcontroller via HTTP requests, environmental data as well as attendance information are sent to the main cloud storage mechanism. ThingSpeak's connectivity lets the system keep data permanently in the cloud and present it graphically so that administrators may remotely track system performance and student attendance.

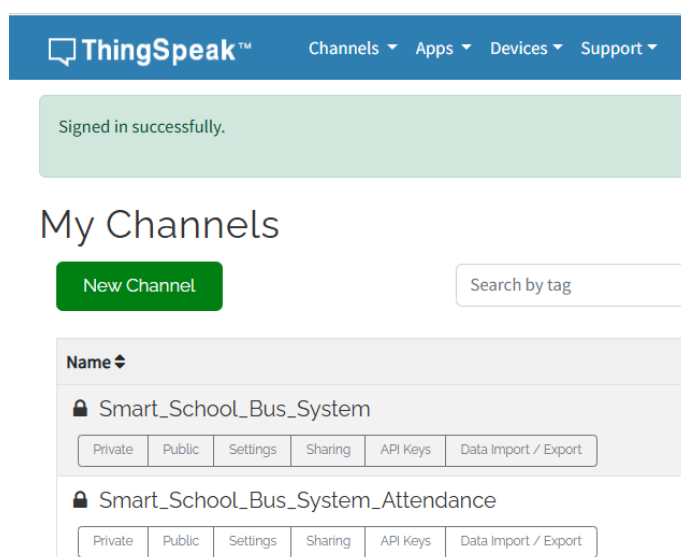


Figure 4.16 ThingSpeak Channels

As stated in Figure 4.16 for this project, two different ThingSpeak channels were made:

a) Environment Monitoring Channel:

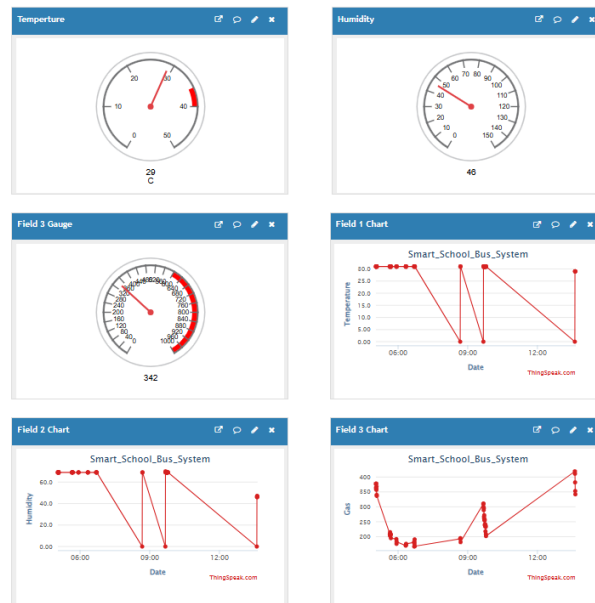


Figure 4.17 Real-time sensor data stored in ThingSpeak Platform

As shown in Figure 4.17 real-time sensor data from the MQ2 gas sensor and the DHT11 temperature and humidity sensor are stored in the environment monitoring channel.

The data fields listed below are utilized as shown in Figure 4.18:

- Field 1: Temperature (°C)
- Field 2: percentage of humidity
- Field 3: Value of the Gas Sensor

Figure 4.18 Channel settings of Environment Monitoring Channel

```
def upload_environment(temp, hum, gas):
    url = (
        f"https://api.thingspeak.com/update"
        f"?api_key={WRITE_API_ENV}"
        f"&field1={temp}"
        f"&field2={hum}"
        f"&field3={gas}"
    )

    try:
        response = urequests.get(url)
        print("Environment Response:", response.text)
        response.close()

    except Exception as e:
        print("Environment Upload Failed", e)
```

Figure 4.19 The upload_environment function in main.py

As shown in Figure 4.19, the upload_environment() function is used by the ESP32 to periodically upload data to ThingSpeak. To prevent going over the API rate restriction, the system makes sure that data is sent every 15 seconds. Real-time monitoring of the bus's interior environment is made possible by the graphical display of the uploaded data. This aids in spotting unusual gas levels or temperatures that can point to dangerous circumstances.

b) Attendance Monitoring Channel



Figure 4.20 Real-time sensor data stored in ThingSpeak Platform

As shown in Figure 4.20, the attendance monitoring channel is used to record student attendance data, route information, and system status updates. This channel plays a key role in tracking student presence during transportation.

The following data fields are used as stated in Figure 4.21:

- Field 4: Student ID
- Field 5: Route (Home to School / School to Home)
- Field 6: Status Code (Present / Duplicate / Unauthorized / Ride End)
- Field 7: Total Present Count
- Field 8: Rejected Count

The screenshot shows the 'Channel Settings' page for a ThingSpeak channel named 'Smart_School_Bus_System_Attendance'. The channel ID is 3409092, the author is mwad000039354889, and the access is Private. The 'Channel Settings' tab is selected, showing a 'Percentage Complete' of 30%. Below this, there are input fields for 'Name' (Smart_School_Bus_System_Attendance) and 'Description'. A list of fields is shown with checkboxes: Field 1, Field 2, Field 3, Field 4 (Student), Field 5 (Route), Field 6 (Status), Field 7 (Present Count), and Field 8 (Rejected Count). Fields 4 through 8 are checked.

Figure 4.21 Channel settings of Attendance Monitoring Channel

```
def upload_attendance(student, route, status_code, present_count, rejected_count):

    url = (
        f"https://api.thingspeak.com/update"
        f"?api_key={WRITE_API_ATT}"
        f"&field4={student}"
        f"&field5={route}"
        f"&field6={status_code}"
        f"&field7={present_count}"
        f"&field8={rejected_count}"
    )

    try:
        response = urequests.get(url)

        print("Attendance Response:", response.text)

        response.close()

    except Exception as e:
        print("Attendance Upload Failed", e)
```

Figure 4.22 The upload_attendance function in main.py

Every time a student scans their RFID card, the ESP32 uses the `upload_attendance()` function to upload attendance data. For monitoring purposes, the system also logs attempts at illegal access and duplicate entries. Administrators may monitor attendance trends and ride statistics in real time thanks to the visualization of this data in ThingSpeak dashboards.

Data Transmission Process

Procedure for Data Transmission

HTTP GET requests are used to transfer data between the ESP32 and ThingSpeak. Before transmitting it to the ThingSpeak server, the ESP32 creates a URL with the sensor values and API key. ThingSpeak provides a response code that indicates a successful data upload when the request has been successfully completed. This attests to the system's proper connection to the cloud platform.

An example of a response seen during testing

- HTTP Reaction: 200 OK
- Data on the Channel Has Been Updated Successfully

This shows that the system can communicate with the cloud in real time without losing any data.

The Smart School Bus Monitoring and Attendance System's usefulness is greatly improved with the integration of ThingSpeak. It enhances monitoring effectiveness and decision-making by offering real-time visualization of environmental variables and attendance data. The system exhibits consistent Wi-Fi connectivity and dependable API communication by uploading sensor data and attendance records without any delays.

Trends like temperature shifts, variations in gas levels, and patterns of student attendance can be easily interpreted thanks to the graphical data display. All things considered, ThingSpeak plays a crucial role in making IoT-based cloud monitoring possible for the suggested system.

4.3.4 Database Design and Development

The Smart School Bus Monitoring and Attendance System uses a lightweight database structure to handle student identification, user authentication, and attendance tracking. The system simulates database capability within the ESP32 microcontroller by using organized Python dictionaries instead of a conventional relational database management system like MySQL. For embedded IoT devices, this method guarantees quick data access, little memory use, and effective real-time processing.

A. Database for user authentication

```
# ===== USERS =====  
DRIVER_UID = "E9BA326D0C"  
AUTHORIZED = {  
    "37C61D49A5": 1,  
    "250D9FE057": 201,  
    "07C8059F55": 251,  
    "299194113D": 258,  
    "FD64F4EC81": 322  
}  
LCD_NAMES = {1: "Student 001", 201: "Student 201", 251: "Student 251", 258: "Student 258", 322: "Student 322"}
```

Figure 4.23 Authorized RFID Database

The main database framework keeps approved RFID card numbers connected to driver and student identities. Every RFID Unique Identifier (UID) is linked to a student ID as shown in Figure 4.23. Using a Python dictionary called AUTHORIZED, this framework is put into action.

Structure Sample:

- RFID UID to Student ID Correspondence During RFID scanning, this database helps to confirm whether a scanned card belongs to a permitted user.
- The system records attendance if a match is discovered; otherwise, it sets off an unauthorized access alert.

B. Database of Student Information

```
# Local student data database matching script.js
STUDENT_DB = {
  "1": "Muhammad Ali",
  "201": "JeyaKumar Saravanan",
  "251": "Joseph Vijay",
  "258": "Sri",
  "322": "Adriana Sarah Binti Ahmad Faiza"
}
```

Figure 4.24 Student Information Database

As shown in Figure 4.24, student names, depending on their assigned student ID, are kept in a secondary database called STUDENT_DB. This enables the system to show pertinent user data instead of just numerical IDs. This database helps user friendliness in:

- Messages shown on an LCD display,
- Notifications for Telegram Reports on ride summaries

C. Storage for Attendance Records

The system keeps attendance information in a list structure called attendance for a short period of time during operation. Every time a student effectively scans their RFID card, their UID is added to the list.

This design enables the system:

- Count current students.
- Stay away from duplicate entries.
- Create ride summary reports.
- A counter variable called rejected is also used to note duplicate scans and unapproved access attempts.

D. Flow of Database Operations

The real-time operation of the database system follows this process: The student scans the RFID card. ESP32 reads the RFID reader's UID. The UID is checked against the AUTHORIZED database.

If correct:

- Student ID is gotten
- The name of a student is retrieved from STUDENT_DB.
- Attendance is noted.

If not valid:

- Access is forbidden.
- Alert is launched.
- This real-time check guarantees that only authorized users may record attendance in the system.

For embedded IoT systems, the implemented database structure offers a lightweight and effective solution. The dictionary-based method is adequate for small-scale applications like school bus surveillance, even if it doesn't make use of external database servers. RFID cards, student identities, and attendance records are all accurately mapped by the system. Additionally, it guarantees quick processing, which is essential for detecting attendance in real time. All things considered, the database design successfully satisfies the system requirements while preserving dependability and simplicity.

4.3.4 Web Dashboard Development

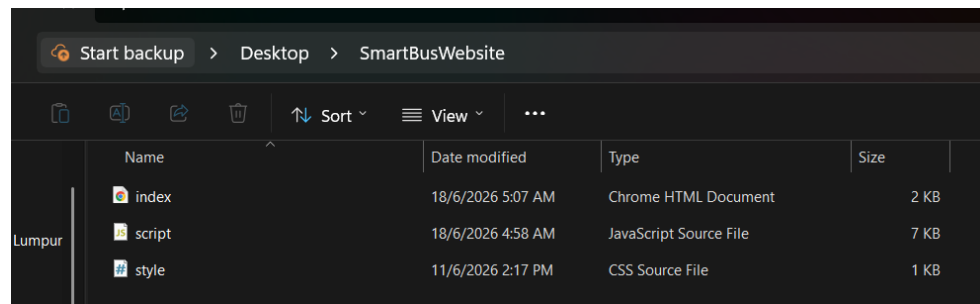


Figure 4.25: The files used to create the web dashboard development

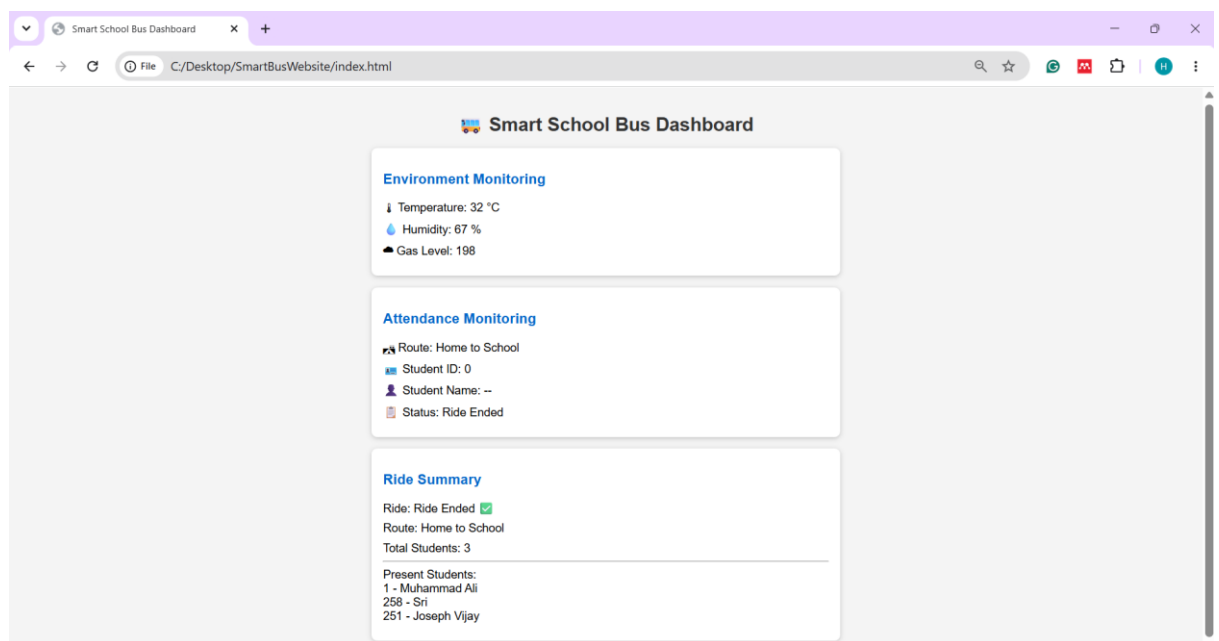


Figure 4.26 Smart School Bus's Web Dashboard

As in Figure 4.25, JavaScript, HTML, and CSS were used in the design of the web dashboard. To enhance use and data readability, it is divided into several sections.

The primary sections consist of:

Environmental Monitoring Section:

As shown in Figure 4.26, the environmental monitoring section displays real-time sensor data collected from the DHT11 temperature and humidity sensor and the MQ2 gas sensor.

The dashboard visualizes:

- Temperature trends (°C)
- Humidity levels (%)
- Gas concentration levels

Graphical representation is used to clearly display environmental changes over time. This helps in identifying abnormal conditions that may affect passenger safety.

Attendance Monitoring Section:

As shown in Figure 4.26, the attendance monitoring interface displays real-time student attendance records captured through RFID scanning.

The system shows:

- Route Information
- Student ID
- Student Name
- Attendance Status (Present / Duplicate / Unauthorized)

This section helps administrators track student boarding activity during each trip. The data is automatically updated whenever a student scans their RFID card, ensuring accurate and up-to-date attendance records.

Ride Summary Section:

As shown in Figure 4.26, the ride summary section displays the values only when the driver presses the end ride button. The dashboard displays:

- Ride:
- Route
- Total Students
- Present Students

Alerts are triggered automatically by the ESP32 and transmitted to the dashboard through cloud integration. This feature enhances system safety by providing immediate visibility of critical events.

The Smart School Bus Monitoring and Attendance System's accessibility and ease of use are greatly enhanced by the web dashboard. The system is appropriate for IoT-based remote applications since users can track real-time data from any place. ThingSpeak, the ESP32, and the web dashboard's integration guarantee seamless data flow from device to cloud to user interface. The dashboard shows live changes right away, proving the system works well. All things considered, the web dashboard improves the system's performance by giving a single, dynamic monitoring platform for environmental and attendance information.

4.3.6 Telegram Notification Integration

The Telegram notification system was added to the Smart School Bus Monitoring and Attendance System to provide real-time updates and notifications to parents and management. This feature makes real-time alerts about ride status, attendance events, and safety concerns available, therefore enhancing communication. The Telegram Bot API was the link between the Telegram messenger platform and the ESP32 microcontroller, as shown in Figure 4.27. The telegram API then shares the message to a preconfigured chat group.



Figure 4.27 Telegram Bot API linked with a Telegram group chat

The system produces many kinds of alerts depending on real-time events:

- Confirmation of student attendance

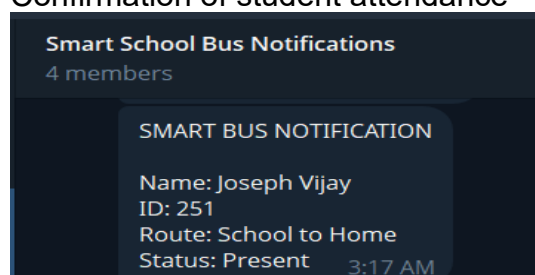


Figure 4.28 Telegram Attendance Notification

- finding duplicate entries

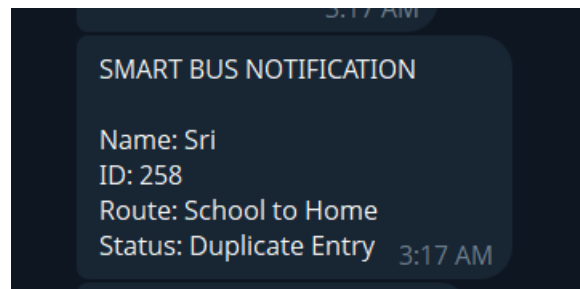


Figure 4.29 Telegram Duplicate entries Notification

- unauthorized access warning

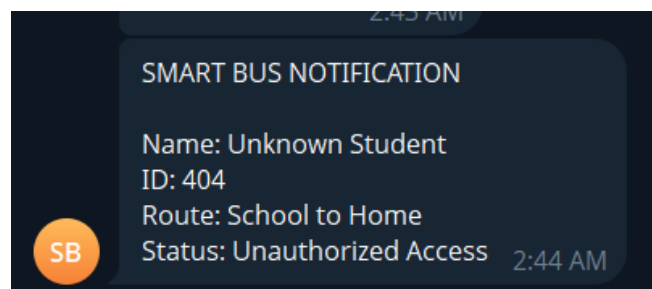


Figure 4.30 Telegram Unauthorized Access Notification

- Notification to begin riding

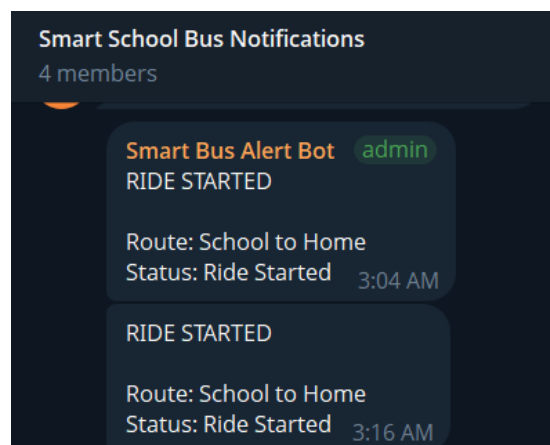


Figure 4.31 Telegram Ride Started Notification

- Ride End Summary

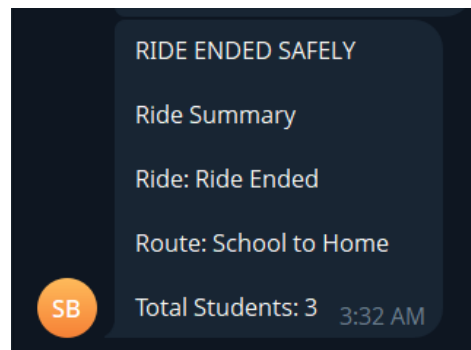


Figure 4.32 Telegram Ride Summary Notification

This mechanism makes sure every stakeholder gets fast changes on bus operations and student attendance.

The Smart School Bus Monitoring and Attendance System's responsiveness and efficacy are greatly enhanced by the incorporation of Telegram notifications. It makes it possible for people and the system to communicate in real time without the need for manual dashboard or cloud platform checks. This guarantees that interested parties are kept up to date on bus operations and student safety.

Overall, Telegram integration adds a dependable and effective communication channel for real-time notifications, improving the system's IoT capacity.

4.4 System Testing and Results

The Smart School Bus Monitoring and Attendance System is examined in this part for testing and assessment. To check the operation, dependability, and performance of all integrated hardware and software parts, system testing is meant. Every component of the system was tested separately and in an integrated setting to guarantee proper operation under real-time situations.

The testing procedure consists of RFID attendance verification, driver authentication, environmental sensor testing, alert system validation, cloud data transmission testing, and web dashboard monitoring evaluation.

4.4.1 RFID Attendance Testing

Table 8 and Figures 3.3, 3.4, and 3.5 explain that, to confirm the accuracy of student identification and attendance recording, RFID testing was carried out. Every approved RFID card was scanned several times to guarantee correct detection and system response. The method correctly recognised valid RFID cards and only once noted attendance per student to stop duplicate entries. Unapproved RFID cards were returned, and suitable warnings were produced.

Table 8: RFID Attendance Testing Results

Test Case	Expected Result	Actual Result	Status
Valid Student Scan	Attendance recorded	Attendance recorded	Pass
Duplicate RFID Scan	Duplicate entry detected	Duplicate detected	Pass
Invalid RFID Scan	Unauthorized Access	Unauthorized Access	Pass

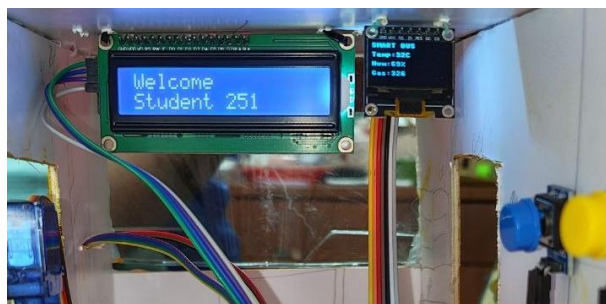


Figure 4.33 Valid Student RFID Scan



Figure 4.34 Duplicate RFID Detection



Figure 4.35 Unauthorized RFID Detection

4.4.2 Driver Authentication Testing

Driver authentication testing was performed to ensure that only authorized drivers could activate the system and start the ride session. Table 9, Figures 4.36 and 4.37 state that the system verifies the driver's RFID card before allowing access to system functions such as route selection and ride initiation

Table 9: Driver Authentication Testing Results

Test Case	Expected Result	Actual Result	Status
Valid Driver Card	Driver Verified	Driver Verified	Pass
Invalid Driver Card	Invalid Driver	Invalid Driver	Pass



Figure 4.36 Successful Driver Authentication



Figure 4.37 Invalid Driver Card Detection

4.4.3 Temperature Sensor Testing

The DHT11 sensor was tested under different temperature conditions to evaluate its accuracy and response to threshold limits. When temperature values exceeded the predefined threshold, the system triggered an alert through the buzzer and LCD

Table 10: Temperature Sensor Testing Results

Temperature (°C)	Expected Result	Actual Result	Status
28°C	Normal operation	Normal operation	Pass
38°C	Warning alert triggered	Alert triggered	Pass

```
last_upload = time.time()
if temp >= 38 and not temp_alert:
    temp_alert = True; beep(5)
    lcd_message("HIGH TEMP!", str(temp) + " C"); time.sleep(3)
elif temp < 32: temp_alert = False
```

Figure 4.38 Coding of High Temperature function



Figure 4.39 High Temperature Alert

4.4.4 Gas Sensor Testing

The MQ2 gas sensor was evaluated to find the gas and smoke levels within the system environment. To guarantee passenger safety, the system triggered visual and audio warnings when gas levels went over the safety limit.

Table 11: Gas Sensor Testing Results

MQ2 Value	Expected Result	Actual Result	Status
200	Normal condition	Normal	Pass
500	Warning alert	Warning alert triggered	Pass

```

if gas >= 500 and not gas_alert:
    gas_alert = True; beep(5)
    lcd_message("! DANGER:", "AIR !"); time.sleep(3)
elif gas < 500: gas_alert = False

```

Figure 4.40 Coding of High Gas Level function



Figure 4.41 High Gas Level Alert

4.4.5 Alert System Testing

The alert system was evaluated based on buzzer activation, LCD messages, OLED warnings, and Telegram notifications.

The system successfully generated alerts for:

- High temperature conditions
- Gas leakage detection
- Unauthorized access attempts
- Duplicate RFID scans

All alerts were triggered in real time, ensuring immediate system response, as stated in Table 12.

Table 12: Alert System Testing Results

Alert Condition	Expected Result	Actual Result	Status
High Temperature	Buzzer activated and warning displayed	Buzzer activated and warning displayed	Pass
High Gas Level	Buzzer activated, and danger alert displayed	Buzzer activated and danger alert displayed	Pass
Unauthorized RFID Scan	Access denied and warning displayed	Access denied and warning displayed	Pass
Duplicate RFID Scan	Duplicate entry warning displayed	Duplicate warning displayed	Pass

4.4.6 Cloud Data Upload Testing

Cloud integration testing was performed to verify data transmission to the ThingSpeak platform, as shown in Table 13 and Figures 4.42 and 4.43. The ESP32 successfully uploaded environmental and attendance data using HTTP requests. The system demonstrated stable Wi-Fi connectivity and consistent data transmission without packet loss.

Table 13: Cloud Data Upload Testing Results

Test Case	Expected Result	Actual Result	Status
Environment Data Upload	Data saved in cloud	Data saved	Pass
Attendance Data Upload	Data recorded	Data recorded	Pass
API Response Check	HTTP 200 OK	HTTP 200 OK	Pass

```

-----
Ride Start Response:
{"ok":true,"result":{"message":
ns","type":"supergroup"},"d
Environment Response: 2102
Environment Response: 2103
Environment Response: 2104

```

Figure 4.42 Serial monitor Output on Environment Data Upload

```

{"ok":true,"result":{"message_id":56,"from":{"id":883814063,"type":"supergroup"},"date":1782587862,"text":"SMART BI
Telegram Notification Sent

===== SUMMARY =====
Route: School to Home
Present: 4
Rejected: 1
Student 258
Student 001
Student 201
Student 251
Waiting before summary upload...
Attendance Response: 82
Ride Summary Notification Sent
Telegram Error: 'NoneType' object has no attribute 'close'
Summary uploaded

```

Figure 4.43 Serial Monitor Output on Ride Summary Data Upload

```

Sending Telegram Notification...
Telegram Response:
{"ok":true,"result":{"message_id":55,"from":{"id":883814063,"type":"supergroup"},"date":1782587842,"text":"SMART BI
Telegram Notification Sent

```

Figure 4.44 Serial Monitor Output on API

4.4.7 Dashboard Monitoring Testing

The web dashboard was tested to ensure the correct retrieval and display of real-time data from the cloud platform. The system successfully displayed attendance records, environmental readings, and alerts. The dashboard updated dynamically without manual refresh, confirming proper API integration between the cloud and frontend system.

Table 14: Web Dashboard Monitoring Testing Results

Test Case	Expected Result	Actual Result	Status
Attendance Display	Live updates	Live updates	Pass
Sensor Graph Display	Accurate graphs	Accurate graphs	Pass

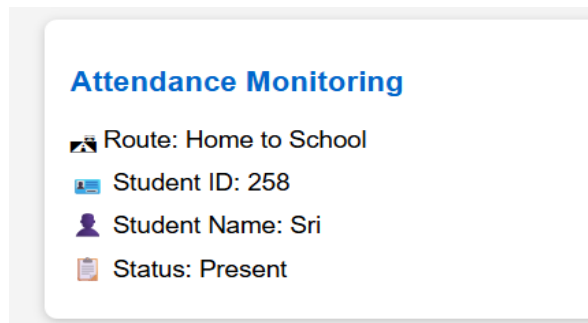


Figure 4.46 Dashboard Attendance Update

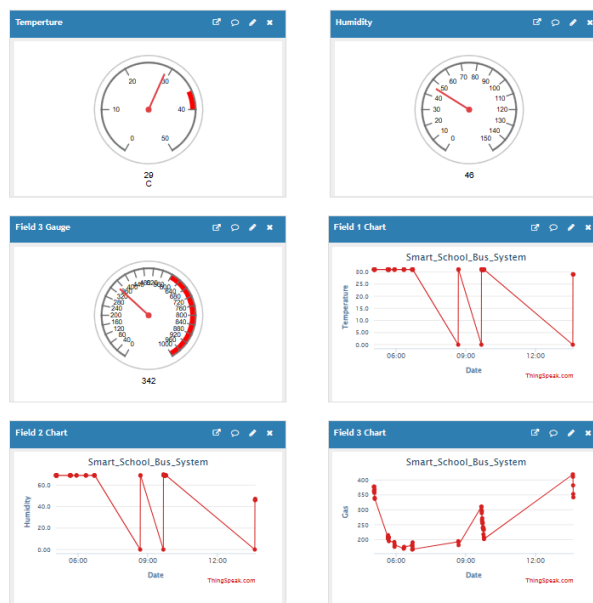


Figure 4.47 Sensor graph display

4.5 Discussion

This section discusses the performance of the Smart School Bus System based on the testing results presented in the previous section. This section focuses on system performance; limitations identified during system development and testing.

4.5.1 System Performance

The test findings show that the Smart School Bus Monitoring and Attendance System ran as planned and met functional criteria. To guarantee ongoing real-time system operation, the ESP32 microcontroller effectively coordinated all hardware components,

including the RFID module, environmental sensors, display units, buzzer, servo motor, and communication modules. The RFID-based attendance system correctly recognized authorized users via distinctive RFID card IDs. The system could record real-time attendance, stop duplicate entries, and forbid unapproved access attempts. This increased the integrity and dependability of the attendance statistics gathered during bus operations.

Using the DHT11 temperature and humidity sensor and MQ2 gas sensor, the environmental monitoring subsystem effectively delivered real-time environmental data to the ESP32. While simultaneously uploading the temperature, humidity, and gas concentration levels to the ThingSpeak cloud platform, the system constantly tracked them and showed the readings on the OLED screen.

The deployed alert system reacted properly under unusual circumstances. The system turned the buzzer on and showed caution notes on the LCD and OLED panels when temperature or gas levels surpassed set limits. Furthermore, send Telegram warnings right away to let consumers know about serious situations. Using ThingSpeak to combine cloud services made the real-time data flow and display consistent. Sensor readings and attendance information were effectively submitted without major lag, therefore assuring constant monitoring capability.

Moreover, the web dashboard offered a good user interface for showing alert status, environmental data, and attendance records. The system showed consistent performance throughout continuous operation, therefore validating its applicability for real-world smart transportation needs.

All things considered, the testing results show that the created system is dependable, effective, and meant to run real-time monitoring, authentication, and alerting capabilities.

4.5.2 Achievement of Project Objectives

The project objectives were assessed using the results of testing shown throughout this chapter, as well as the execution results.

Table 15 Lists the project Objectives and Achievement Status

Project Objective	Achievement Status
To develop an automated student attendance system using RFID-based authentication	Achieved
To design a web-based dashboard for real-time monitoring accessible by parents and administrators	Achieved
To implement IoT-based real-time monitoring of temperature, humidity, and gas levels using ThingSpeak	Achieved

Analysis of Achieved Objectives

The first goal was effectively met by use of an RFID-based authentication method with the ESP32 microcontroller with the MFRC522 module built in. The system could correctly spot authorized students and instantly record attendance while forbidding duplicate entries and unauthorized access.

A web-based dashboard showing real-time attendance statistics, environmental readings, and system alerts helps to reach the second goal. The dashboard offers a simple interface that lets parents and school administrators watch bus activity from afar quickly.

The ThingSpeak Internet of Things platform—which allows ongoing transmission and display of environmental sensor data like temperature, humidity, and gas levels—was integrated to help to meet the third goal. The technology effectively uploads data in real time, so enabling historical data analysis and distant monitoring.

All of the project goals stated in Chapter 1 were generally effectively met. The findings support the Smart School Bus Monitoring and Attendance System's excellent performance as an IoT-based monitoring and safety solution for school transport networks.

4.5.3 Limitations of the system

Although the Smart School Bus Monitoring and Attendance System met its intended goals, development and testing revealed several restrictions. The system first relies on a reliable Wi-Fi connection to talk to the Telegram notification service and the ThingSpeak cloud platform. Any disturbance in the online connection could compromise notification delivery, cloud updates, and real-time data flow. Parents and administrators might therefore have delays getting attendance information, environmental monitoring data, and system warnings during network outages. The technology was also tested in a prototype-scale setting instead of an actual school bus running under actual traffic situations. Though the test findings showed good performance within the prototype environment, more field testing in actual transportation environments would help to assess long-term performance, toughness, and dependability. Moreover, the present system lacks a GPS tracking capability. Parents and administrators cannot follow the school bus's real-time position. At present, the system is constrained to notifications, environmental monitoring, and attendance tracking. Furthermore, student identification is dependent only on RFID authentication. Though RFID technology offers accurate attendance confirmation, the technology does not include biometric authentication techniques like facial recognition. If a card is lost, shared, or utilized by another person, the likelihood of RFID card abuse cannot be totally avoided. Notwithstanding these restrictions, the created system effectively shows the viability of combining RFID technology, ecological monitoring sensors, cloud computing, and IoT communication technologies into a smart school bus monitoring system. The findings show that the approach offers a strong basis for enhancing student attendance control and vehicle safety in educational settings.

CHAPTER 5: CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This chapter concludes the study by summarising the overall findings and achievements of the system. It discusses the extent to which the project objectives were achieved and presents several recommendations for future improvements and enhancements to the proposed system.

The project focused on the development of an IoT-based school bus monitoring solution that integrates RFID attendance verification, environmental monitoring sensors, cloud-based data storage, a web dashboard, and Telegram notification services. The effectiveness of the developed system was evaluated through a series of hardware and software testing activities presented in Chapter 4.

5.2 Conclusion

The Smart School Bus Monitoring and Attendance System was successfully designed, developed, and evaluated using the ESP32 microcontroller, RFID RC522 reader, DHT11 temperature and humidity sensor, MQ2 gas sensor, OLED display, LCD display, servo motor, buzzer, ThingSpeak cloud platform, Telegram notification service, and a web-based dashboard.

The system was created to offer a sensible and reasonably priced way to keep track of conditions inside a school transit environment and student attendance. The system helps to improve transportation safety by means of ongoing environmental monitoring and automatically records student attendance by combining RFID authentication with IoT technologies.

The RFID module successfully recognized authorized students and drivers, precisely noted attendance, and stopped duplicate attendance entries and unwanted access attempts, therefore helping to improve security. Parents and administrators could see the attendance data in real time on a web-based dashboard that was sent to the cloud platform.

Inside the bus surroundings, the environmental monitoring subsystem effectively detected temperature, humidity, and gas concentration levels. When predefined safety thresholds were exceeded, the system activated warning alerts through the buzzer and display units while simultaneously transmitting notifications through Telegram. This functionality contributes to improving passenger safety during transportation activities.

The integration of ThingSpeak cloud services enabled reliable storage and visualization of environmental and attendance data. Furthermore, the Telegram notification system provided real-time updates regarding attendance status, ride commencement, ride completion, and unauthorized access attempts. The web dashboard served as a centralized monitoring platform that enhanced accessibility and user interaction.

The testing and evaluation activities confirmed that all hardware and software components operated according to their intended functions. RFID testing, sensor functionality testing, alert verification testing, cloud communication testing, and dashboard monitoring testing demonstrated that the system performed reliably within the scope of the prototype environment.

Overall, the project successfully achieved all of the objectives defined in Chapter 1. The developed Smart School Bus Monitoring and Attendance System demonstrates the feasibility of integrating RFID technology, IoT communication, cloud computing, and real-time notification services into a single transportation monitoring platform. The system provides an effective foundation for improving student safety, attendance management, and transportation monitoring in educational institutions.

5.3 Future Recommendations

Although the Smart School Bus Monitoring and Attendance System successfully achieved its intended objectives, several improvements may be considered for future development.

First of all, the system can include a Global Positioning System (GPS) module that lets you keep track of where the bus is in real time. This improvement would enable school officials and parents to more precisely track the bus's present location and project arrival times.

Secondly, a more sophisticated database management system—such as MySQL or Firebase—may take the place of the existing local database setup. This would enhance user management, long-term record keeping, and data storage capacity for bigger installations with numerous buses and kids.

Future development may also include the implementation of facial recognition technology alongside RFID authentication. This would provide an additional layer of security by verifying student identity through biometric recognition and reducing the possibility of unauthorized card usage.

Finally, the system should be tested and deployed within actual school bus environments for extended periods. Real-world deployment would allow the evaluation of long-term reliability, scalability, and performance under varying transportation conditions.

5.4 Chapter Summary

The conclusion and recommendations for the Smart School Bus Monitoring and Attendance System were given in this chapter. The initiative effectively showed how RFID technology, ecological monitoring sensors, cloud computing technologies, and real-time notification systems can be combined into one IoT-based transportation monitoring solution.

The created system could automatically track student attendance, keep an eye on environmental factors, send out warning warnings, upload information to cloud services, and offer distant access via a web dashboard and Telegram

notifications. The testing findings showed that the system ran as planned and effectively fulfilled all of the project's goals.

Also put forth were several ideas for future changes, such mobile application development, predictive analytics, and real-world deployment, GPS tracking integration, sophisticated database implementation, more environmental sensors, facial recognition technology. These suggestions present chances to enhance the system's practical use, scalability, and practicality in next developments.

All told, the Smart School Bus Monitoring and Attendance System shows how IoT technology can improve student safety, attendance management, and vehicle monitoring inside classroom settings.

REFERENCES

1. *CSCA0101 Computing Basics Chapter 3 Input Devices*. (n.d.).
2. *ESP32 Wi-Fi & Bluetooth SoC | Espressif Systems*. (n.d.). Retrieved June 25, 2026, from <https://www.espressif.com/en/products/socs/esp32>
3. *images (300×168)*. (n.d.). Retrieved December 9, 2025, from https://encrypted-tbn1.gstatic.com/images?q=tbn:ANd9GcRTBO9kXFyadvF0dBvIWrr--Ardub_258CWAvIA5CPEXh0WWc39I
4. Krishnan, R. S., Kannan, A., Manikandan, G., Kb, S. S., Sankar, V. K., & Narayanan, K. L. (2021). Secured college bus management system using IoT for Covid-19 pandemic situation. *Proceedings of the 3rd International Conference on Intelligent Communication Technologies and Virtual Mobile Networks, ICICV 2021*. <https://doi.org/10.1109/ICICV50876.2021.9388378>
5. Li, C., Nguyen, Q., Ryan, P. H., Lemasters, G. K., Spitz, H., Lobaugh, M., Glover, S., & Grinshpun, S. A. (2009). School bus pollution and changes in the air quality at schools: A case study. *Journal of Environmental Monitoring*, 11(5). <https://doi.org/10.1039/b819458k>
6. Nguyen, V. D., Khoa, H. Van, Kieu, T. N., & Huh, E. N. (2022). Internet of Things-Based Intelligent Attendance System: Framework, Practice Implementation, and Application. *Electronics (Switzerland)*, 11(19). <https://doi.org/10.3390/electronics11193151>
7. *Output Devices - GeeksforGeeks*. (n.d.). Retrieved June 25, 2026, from <https://www.geeksforgeeks.org/computer-science-fundamentals/what-are-different-output-devices/>
8. Qu, J., Zheng, L., Kang, Y., & Ma, B. (2022). Design and Implementation of Teaching Assistant System for Mechanical Course based on Mobile AR Technology. *International Journal of Advanced Computer Science and Applications*, 13(4). <https://doi.org/10.14569/IJACSA.2022.0130410>
9. Raj, J. T., & Sankar, J. (2018). IoT based smart school bus monitoring and notification system. *5th IEEE Region 10 Humanitarian Technology Conference 2017, R10-HTC 2017, 2018-January*. <https://doi.org/10.1109/R10-HTC.2017.8288913>
10. *Smart School Bus System | FMS Tech - Fleet Management System*. (n.d.). Retrieved November 13, 2025, from <https://fms-tech.com/solutions/smart-school-bus-system/>
11. *Telegram Bot API*. (n.d.). Retrieved November 26, 2025, from <https://core.telegram.org/bots/api>
12. Todorov, A., Gicheva, P., Stoykova, V., Karapetkov, S., Uzunov, H., Dechkova, S., & Zlatev, Z. (2023). Environmental Monitoring in Bus

- Transportation Using a Developed Measurement System. *Urban Science*, 7(3). <https://doi.org/10.3390/urbansci7030090>
13. *Top 12 Software Development Methodologies - TatvaSoft Blog*. (n.d.). Retrieved November 27, 2025, from <https://www.tatvasoft.com/blog/top-12-software-development-methodologies-and-its-advantages-disadvantages/>
 14. *Wokwi - Online ESP32, STM32, Arduino Simulator*. (n.d.). Retrieved November 26, 2025, from <https://wokwi.com/projects/431216610309769217>
 15. Yang, H., & Han, X. (2020). Face recognition attendance system based on real-time video processing. In *IEEE Access* (Vol. 8). <https://doi.org/10.1109/ACCESS.2020.3007205>

APPENDIX A: SOURCE CODE

Main.py

(the main coding for the smart school bus system)

```
import time
from machine import Pin, SPI, SoftI2C, SoftSPI, PWM, ADC
from mfrc522 import MFRC522
from i2c_lcd import I2cLcd
from ssd1306 import SSD1306_SPI
import dht
import network
import urequests

# ===== RFID =====
spi = SPI(1, baudrate=1000000, polarity=0, phase=0,
          sck=Pin(18), mosi=Pin(23), miso=Pin(19))
rdr = MFRC522(spi, Pin(5, Pin.OUT))

# ===== LCD =====
i2c = SoftI2C(scl=Pin(22), sda=Pin(21))
lcd = I2cLcd(i2c, 0x27, 2, 16)

# ===== OLED =====
spi_oled = SoftSPI(baudrate=1000000, polarity=0, phase=0,
                   sck=Pin(14), mosi=Pin(13), miso=Pin(12))
oled = SSD1306_SPI(128, 64, spi_oled, dc=Pin(17), res=Pin(27), cs=Pin(26))

# ===== BUTTONS =====
btnA = Pin(32, Pin.IN)
btnB = Pin(35, Pin.IN)
btnC = Pin(2, Pin.IN, Pin.PULL_UP)

# ===== BUZZER =====
buzzer = Pin(16, Pin.OUT)

# ===== SERVO =====
servo = PWM(Pin(33), freq=50)

# ===== DHT11 =====
dht_sensor = dht.DHT11(Pin(25))

# ===== MQ2 =====
mq2 = ADC(Pin(34))
mq2.atten(ADC.ATTN_11DB)
```

```

# ===== USERS =====
DRIVER_UID = "E9BA326D0C"
AUTHORIZED = {
    "37C61D49A5": 1,
    "250D9FE057": 201,
    "07C8059F55": 251,
    "299194113D": 258,
    "FD64F4EC81": 322
}
LCD_NAMES = {1: "Student 001", 201: "Student 201", 251: "Student 251", 258: "Student 258", 322: "Student 322"}

attendance = []
rejected = 0
route = 0
route_name = ""
temp_alert = False
gas_alert = False
last_upload = 0

# ===== THINGSPEAK =====

# Environment Channel
WRITE_API_ENV = "5XFHAJWQFIMYMH5F"

# Attendance Channel
WRITE_API_ATT = "F9EDOZV0NNQESNOW"

# ===== TELEGRAM CONFIGURATION =====
BOT_TOKEN = "8838140634:AAG4T6DhIliRmJSnWTr2FdM7dDKI3-2jPGk"
CHAT_ID = "-1003739885614"

# Local student data database matching script.js
STUDENT_DB = {
    "1": "Muhammad Ali",
    "201": "JeyaKumar Saravanan",
    "251": "Joseph Vijay",
    "258": "Sri"
}

```

```

        "258": "Sri",
        "322": "Adriana Sarah Binti Ahmad Faiza"
    }

# ===== TELEGRAM FUNCTION =====
def send_telegram_alert(student_id, route_code, status_code):

    try:

        student_name = STUDENT_DB.get(str(student_id), "Unknown Student")

        if route_code == 1:
            route_text = "Home to School"
        elif route_code == 2:
            route_text = "School to Home"
        else:
            route_text = "Unknown Route"

        if status_code == 1:
            status_text = "Present"
        elif status_code == 2:
            status_text = "Duplicate Entry"
        elif status_code == 3:
            status_text = "Unauthorized Access"
        elif status_code == 9:
            status_text = "Ride Ended"
        else:
            status_text = "Unknown"

        message = (
            "SMART BUS NOTIFICATION\n\n"
            "Name: {}\n"
            "ID: {}\n"
            "Route: {}\n"
            "Status: {}"
        ).format(
            student_name,
            student_id,
            route_text,
            status_text
        )

```

```

    )

    url = "https://api.telegram.org/bot{}/sendMessage".format(BOT_TOKEN)

    payload = {
        "chat_id": CHAT_ID,
        "text": message
    }

    print("Sending Telegram Notification...")

    response = urequests.post(url, json=payload)

    print("Telegram Response:")
    print(response.text)

    response.close()

    print("Telegram Notification Sent")

except Exception as e:
    print("Telegram Error:", e)
# ===== FUNCTIONS =====

def connect_wifi():
    ssid = "RMHPS-2.4"
    password = "0530252218"
    wlan = network.WLAN(network.STA_IF)
    wlan.active(False); time.sleep(1)
    wlan.active(True); time.sleep(1)
    print("Connecting WiFi...")
    wlan.connect(ssid, password)
    timeout = 15; start = time.time()
    while not wlan.isconnected():
        if time.time() - start > timeout:
            print("WiFi Connection Failed"); return
        time.sleep(0.5)
    print("WiFi Connected"); print(wlan.ifconfig())

def lcd_message(line1="", line2=""):
    lcd.clear(); lcd.outstr(line1 + "\n" + line2)

```

```

def lcd_message(line1="", line2=""):
    lcd.clear(); lcd.putstr(line1 + "\n" + line2)

def beep(times):
    for i in range(times):
        buzzer.on(); time.sleep(0.2)
        buzzer.off(); time.sleep(0.2)

def open_door(): servo.duty(40)
def close_door(): servo.duty(115)

def upload_environment(temp, hum, gas):
    url = (
        f"https://api.thingspeak.com/update"
        f"?api_key={WRITE_API_ENV}"
        f"&field1={temp}"
        f"&field2={hum}"
        f"&field3={gas}"
    )

    try:
        response = urequests.get(url)
        print("Environment Response:", response.text)
        response.close()

    except Exception as e:
        print("Environment Upload Failed", e)

def upload_attendance(student, route, status_code, present_count, rejected_count):
    url = (
        f"https://api.thingspeak.com/update"
        f"?api_key={WRITE_API_ATT}"
        f"&field4={student}"
        f"&field5={route}"
        f"&field6={status_code}"
        f"&field7={present_count}"
    )

```

```

        f"&field7={present_count}"
        f"&field8={rejected_count}"
    )

    try:
        response = urequests.get(url)

        print("Attendance Response:", response.text)

        response.close()

    except Exception as e:
        print("Attendance Upload Failed", e)

def get_uid():
    (stat, tag_type) = rdr.request(rdr.REQIDL)
    if stat == rdr.OK:
        (stat, uid) = rdr.anticoll()
        if stat == rdr.OK:
            return "".join("{:02X}".format(x) for x in uid)
        return None

def send_ride_started(route_code):

    if route_code == 1:
        route_text = "Home to School"
    elif route_code == 2:
        route_text = "School to Home"
    else:
        route_text = "Unknown Route"

    message = (
        "RIDE STARTED\n\n"
        "Route: {}\n"
        "Status: Ride Started"
    ).format(route_text)

    try:
        url = "https://api.telegram.org/bot{}/sendMessage".format(BOT_TOKEN)

```

```

try:
    url = "https://api.telegram.org/bot{}/sendMessage".format(BOT_TOKEN)

    payload = {
        "chat_id": CHAT_ID,
        "text": message
    }
    response = urequests.post(url, json=payload)

    print("Ride Start Response:")
    print(response.text)

    response.close()

except Exception as e:
    print("Ride Start Error:", e)

def send_ride_summary(route_code, total_students):

    if route_code == 1:
        route_text = "Home to School"
    elif route_code == 2:
        route_text = "School to Home"
    else:
        route_text = "Unknown Route"

    message = (
        "RIDE ENDED SAFELY\n\n"
        "Ride Summary\n\n"
        "Ride: Ride Ended \n\n"
        "Route: {}\n\n"
        "Total Students: {}"
    ).format(
        route_text,
        total_students
    )

    url = "https://api.telegram.org/bot{}/sendMessage".format(BOT_TOKEN)

    payload = {
        "chat_id": CHAT_ID.

```

```

    }

    try:
        response = urequests.post(url, json=payload)
        response.close()
        print("Ride Summary Notification Sent")
        print(response.text)
    except Exception as e:
        print("Telegram Error:", e)

def show_environment():
    global temp_alert, gas_alert, last_upload
    try:
        dht_sensor.measure()
        temp = dht_sensor.temperature()
        hum = dht_sensor.humidity()
        gas = mq2.read()
        if time.time() - last_upload >= 15:
            upload_environment(temp, hum, gas)
            last_upload = time.time()
        if temp >= 38 and not temp_alert:
            temp_alert = True; beep(5)
            lcd_message("HIGH TEMP!", str(temp) + " C"); time.sleep(3)
        elif temp < 32: temp_alert = False
        if gas >= 500 and not gas_alert:
            gas_alert = True; beep(5)
            lcd_message("! DANGER:", "AIR !"); time.sleep(3)
        elif gas < 500: gas_alert = False
        oled.fill(0)
        oled.text("SMART BUS", 0, 0)
        oled.text(f"Temp:{temp}C", 0, 15)
        oled.text(f"Hum:{hum}%", 0, 30)
        oled.text(f"Gas:{gas}", 0, 45)
        oled.show()
    except:
        oled.fill(0); oled.text("Sensor Error", 0, 0); oled.show()

def scan_driver():
    while True:
        lcd_message("Scan Driver's", "Card")
        uid = get_uid()

```

```

uid = get_uid()
if uid:
    print("Detected:", uid)
    if uid == DRIVER_UID:
        print("Driver Verified"); beep(1)
        open_door(); time.sleep(3); close_door()
        lcd_message("Driver", "Verified")
        oled.fill(0); oled.text("Driver Verified", 0, 20); oled.show()
        time.sleep(2); return
    else:
        lcd_message("Invalid", "Driver"); beep(3); time.sleep(2)

def select_route():
    global route, route_name
    lcd_message("Press A/B", "Select Route")

    while True:

        if btnA.value() == 1:
            route = 1
            route_name = "Home to School"
            lcd_message(route_name, "Selected");
            send_ride_started(route)
            time.sleep(2)
            return

        if btnB.value() == 1:
            route = 2
            route_name = "School to Home"
            lcd_message(route_name, "Selected")
            send_ride_started(route)
            time.sleep(2)
            return

def ride_session():
    global rejected
    lcd_message("Ride Started", route_name); time.sleep(2)
    while True:
        show_environment()
        if btnC.value() == 1: return
        uid = get_uid()

```

```

        print("Student UID:", uid)
        if uid in AUTHORIZED:
            student_num = AUTHORIZED[uid]; display_name = LCD_NAMES[student_num]
            if uid not in attendance:
                attendance.append(uid)
                lcd_message("Welcome", display_name)
                open_door(); time.sleep(3); close_door()
                upload_attendance(student_num, route, 1, len(attendance), rejected)
                send_telegram_alert(student_num, route, 1)
            else:
                lcd_message("Duplicate", "Entry"); beep(2); time.sleep(2)
                rejected += 1
                upload_attendance(student_num, route, 2, len(attendance), rejected)
                send_telegram_alert(student_num, route, 2)
        else:
            lcd_message("Unauthorized", "Access"); beep(3); time.sleep(2)
            rejected += 1
            upload_attendance(404, route, 3, len(attendance), rejected)
            send_telegram_alert(404, route, 3)
    time.sleep(0.1)

def ride_summary():
    total = len(attendance)
    lcd_message("Ride Ended", ""); time.sleep(2)
    oled.fill(0)
    oled.text("RIDE SUMMARY", 0, 0)
    oled.text("Route:", 0, 15); oled.text(route_name, 0, 25)
    oled.text("Present:" + str(total), 0, 40)
    oled.text("Reject:" + str(rejected), 0, 55); oled.show()
    print("\n===== SUMMARY =====")
    print("Route:", route_name); print("Present:", total); print("Rejected:", rejected)
    for uid in attendance:
        if uid in AUTHORIZED:
            print(LCD_NAMES[AUTHORIZED[uid]])
    # Upload ride completed marker
    print("Waiting before summary upload...")
    time.sleep(6)
    upload_attendance(
        0,
        route,
        9,

```

```

# Upload ride completed marker
print("Waiting before summary upload...")
time.sleep(6)
upload_attendance(
    0,
    route,
    9,
    total,
    rejected
)
send_ride_summary(route, total)
print("Summary uploaded")
time.sleep(6)

def reset_system():
    global attendance, rejected, route, route_name
    attendance = []
    rejected = 0
    route = 0
    route_name = ""

# ===== EXECUTION LOOP =====
connect_wifi()

while True:
    reset_system()
    scan_driver()
    select_route()
    ride_session()
    ride_summary()

```

Connect_wifi.py

```
#HEMA SAUNDARASAN
#52224123036

#connect_wifi.py
import network
import time

ssid = "RMHPS-2.4"
password = "0530252218"

wlan = network.WLAN(network.STA_IF)

# Reset WiFi properly
wlan.active(False)
time.sleep(1)
wlan.active(True)

print("Connecting to WiFi...")

wlan.connect(ssid, password)

# Wait until connected
timeout = 15 # seconds
start = time.time()

while not wlan.isconnected():
    if time.time() - start > timeout:
        print("Connection FAILED ✖")
        break
    print("Still connecting...")
    time.sleep(1)

if wlan.isconnected():
    print("Connected ✔")
    print("IP:", wlan.ifconfig())
```

I2c_lcd.py

```
#-----HEMA SAUNDARASAN
#-----52224123036

from time import sleep_us, sleep

class I2CLcd:
    def __init__(self, i2c, addr, rows, cols):
        self.i2c = i2c
        self.addr = addr
        self.rows = rows
        self.cols = cols

        self.backlight = 0x08

        sleep(0.05)
        self.init_lcd()

    def write(self, data):
        self.i2c.writeto(self.addr, bytearray([data | self.backlight]))

    def pulse(self, data):
        self.write(data | 0x04)
        sleep_us(1)
        self.write(data & ~0x04)
        sleep_us(50)

    def write4bits(self, data):
        self.write(data)
        self.pulse(data)

    def cmd(self, cmd):
        high = cmd & 0xf0
        low = (cmd << 4) & 0xf0

        self.write4bits(high)
        self.write4bits(low)

    def data(self, char):
        high = (ord(char) & 0xf0) | 0x01
        low = ((ord(char) << 4) & 0xf0) | 0x01

        self.write4bits(high)
        self.write4bits(low)

    def init_lcd(self):
        sleep(0.05)
        self.cmd(0x33)
        self.cmd(0x32)
        self.cmd(0x28)
        self.cmd(0x0c)
        self.cmd(0x80)
        self.clear()

    def clear(self):
        self.cmd(0x01)
        sleep(0.002)

    def putstr(self, string):
        for char in string:
            if char == '\n':
                self.cmd(0xc0)
```

Ssd1306.py

```
#-----HEMA SAUNDARASAN
#-----52224123036

#servo.py

from micropython import const
import framebuffer

SET_CONTRAST = const(0x01)
SET_ENTIRE_ON = const(0x44)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xA4)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA8)
SET_MUX_RATIO = const(0xA8)
SET_COM_OUT_DIR = const(0xC8)
SET_DISP_OFFSET = const(0x03)
SET_COM_PIN_CFG = const(0xDA)
SET_DISP_CLK_DIV = const(0xD5)
SET_PRECHARGE = const(0xD9)
SET_VCOM_DESEL = const(0xDB)
SET_CHARGE_PUMP = const(0x8D)

class SSD1306:
    def __init__(self, width, height, external_vcc):
        self.width = width
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        self.fb = framebuffer.FrameBuffer(self.buffer, self.width, self.height, framebuffer.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP | 0x00,
            SET_MEM_ADDR, 0x00,
            SET_DISP_START_LINE | 0x00,
            SET_SEG_REMAP | 0x01,
            SET_MUX_RATIO, self.height - 1,
            SET_COM_OUT_DIR | 0x08,
            SET_DISP_OFFSET, 0x00,
            SET_COM_PIN_CFG, 0x12,
            SET_DISP_CLK_DIV, 0x80,
            SET_PRECHARGE, 0xF1,
            SET_VCOM_DESEL, 0x30,
            SET_CONTRAST, 0xFF,
            SET_ENTIRE_ON,
            SET_NORM_INV,
            SET_NORM_INV,
            SET_CHARGE_PUMP, 0x14,
            SET_DISP | 0x01,
        ):
            self.write_cmd(cmd)
            self.fill(0)
            self.show()

        def fill(self, col):
            self.fb.fill(col)

        def text(self, string, x, y):
            self.fb.text(string, x, y)

        def show(self):
            self.write_data(self.buffer)

class SSD1306_SPI(SSD1306):
    def __init__(self, width, height, spi, dc, res, cs, external_vcc=False):
        self.spi = spi
        self.dc = dc
        self.res = res
        self.cs = cs

        dc.init(dc.OUT, value=0)
        res.init(res.OUT, value=1)
        cs.init(cs.OUT, value=1)

        self.reset()
        super().__init__(width, height, external_vcc)

    def reset(self):
        self.res(0)
        self.res(1)

    def write_cmd(self, cmd):
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([cmd]))
        self.cs(1)

    def write_data(self, buf):
        self.dc(1)
        self.cs(0)
        self.spi.write(buf)
        self.cs(1)
```

Servo.py

```
#-----HEMA SAUNDARASAN
#-----52224123036

#servo.py

from machine import PWM, Pin

class Servo:
    def __init__(self, pin, freq=50):
        self.pwm = PWM(pin, freq=freq)

    def write_angle(self, angle):
        min_us = 500
        max_us = 2500
        duty = int(min_us + (angle/180)*(max_us-min_us))
        self.pwm.duty_u16(int(duty/20000*65535))
```

Mfrc522.py

```

1 #-----HEM4 SAUNDARANAN
2 #-----5224123036
3
4 #mfrc522.py
5
6 from machine import Pin, SPI
7 from os import uname
8
9 class MFRC522:
10
11     OK = 0
12     NOTAGERR = 1
13     ERR = 2
14
15     REQIDL = 0x26
16     REQALL = 0x52
17     AUTHENT1A = 0x00
18     AUTHENT1B = 0x01
19
20     def __init__(self, spi, cs):
21
22         self.spi = spi
23         self.cs = cs
24         self.cs.value(1)
25         self.spi.init()
26         self.init()
27
28     def _wreg(self, reg, val):
29
30         self.cs.value(0)
31         self.spi.write(b'%c' % int(0xFF & ((reg << 1) & 0x7E)))
32         self.spi.write(b'%c' % int(0xFF & val))
33         self.cs.value(1)
34
35     def _rreg(self, reg):
36
37         self.cs.value(0)
38         self.spi.write(b'%c' % int(0xFF & (((reg << 1) & 0x7E) | 0x80)))
39         val = self.spi.read(1)
40         self.cs.value(1)
41
42         return val[0]
43
44     def _sflags(self, reg, mask):
45         self._wreg(reg, self._rreg(reg) | mask)
46
47     def _cflags(self, reg, mask):
48         self._wreg(reg, self._rreg(reg) & (~mask))
49
50     def _tocard(self, cmd, send):

```

```

101         elif n > 16:
102             n = 16
103
104         for _ in range(n):
105             recv.append(self._rreg(0x09))
106         else:
107             stat = self.ERR
108
109         return stat, recv, bits
110
111     def _crc(self, data):
112
113         self._cflags(0x05, 0x04)
114         self._sflags(0x0A, 0x80)
115
116         for c in data:
117             self._wreg(0x09, c)
118
119         self._wreg(0x01, 0x03)
120
121         i = 0xFF
122         while True:
123             n = self._rreg(0x05)
124             i -= 1
125             if not ((i != 0) and not (n & 0x04)):
126                 break
127
128         return [self._rreg(0x22), self._rreg(0x21)]
129
130     def init(self):
131
132         self.reset()
133         self._wreg(0x2A, 0x8D)
134         self._wreg(0x2B, 0x3E)
135         self._wreg(0x2D, 30)
136         self._wreg(0x2C, 0)
137         self._wreg(0x15, 0x40)
138         self._wreg(0x11, 0x3D)
139         self.antenna_on()
140
141     def reset(self):
142         self._wreg(0x01, 0x0F)
143
144     def antenna_on(self, on=True):
145
146         if on and ~(self._rreg(0x14) & 0x03):
147             self._sflags(0x14, 0x03)
148         else:
149             self._cflags(0x14, 0x03)
150
151     def request(self, mode):

```

```

53         recv = []
54         bits = irq_en = wait_irq = n = 0
55         stat = self.ERR
56
57         if cmd == 0x0E:
58             irq_en = 0x12
59             wait_irq = 0x10
60         elif cmd == 0x0C:
61             irq_en = 0x77
62             wait_irq = 0x30
63
64         self._wreg(0x02, irq_en | 0x80)
65         self._cflags(0x04, 0x80)
66         self._sflags(0x0A, 0x80)
67         self._wreg(0x01, 0x00)
68
69         for c in send:
70             self._wreg(0x09, c)
71             self._wreg(0x01, cmd)
72
73         if cmd == 0x0C:
74             self._sflags(0x00, 0x80)
75
76         i = 2000
77         while True:
78             n = self._rreg(0x04)
79             i -= 1
80             if ~(i != 0) and ~(n & 0x01) and ~(n & wait_irq):
81                 break
82
83         self._cflags(0x00, 0x80)
84
85         if i:
86             if (self._rreg(0x06) & 0x1B) == 0x00:
87                 stat = self.OK
88
89             if n & irq_en & 0x01:
90                 stat = self.NOTAGERR
91             elif cmd == 0x0C:
92                 n = self._rreg(0x0A)
93                 lbits = self._rreg(0x0C) & 0x07
94                 if lbits != 0:
95                     bits = (n - 1) * 8 + lbits
96                 else:
97                     bits = n * 8
98
99             if n == 0:
100                 n = 1
101             elif n > 16:

```

```

102         self._wreg(0x00, 0x07)
103         (stat, recv, bits) = self._tocard(0x0C, [mode])
104
105         if (stat != self.OK) | (bits != 0x10):
106             stat = self.ERR
107
108         return stat, bits
109
110     def anticoll(self):
111
112         ser_chk = 0
113         ser = [0x03, 0x20]
114
115         self._wreg(0x00, 0x00)
116         (stat, recv, bits) = self._tocard(0x0C, ser)
117
118         if stat == self.OK:
119             if len(recv) == 5:
120                 for i in range(4):
121                     ser_chk = ser_chk ^ recv[i]
122                 if ser_chk != recv[4]:
123                     stat = self.ERR
124             else:
125                 stat = self.ERR
126
127         return stat, recv
128
129     def select_tag(self, ser):
130
131         buf = [0x03, 0x70] + ser[1:]
132         buf += self._crc(buf)
133         (stat, recv, bits) = self._tocard(0x0C, buf)
134         return self.OK if (stat == self.OK) and (bits == 0x18) else self.ERR
135
136     def auth(self, mode, addr, sect, ser):
137         return self._tocard(0x0E, [mode, addr] + sect + ser[1:])[0]
138
139     def stop_crypto1(self):
140         self._cflags(0x08, 0x08)
141
142     def read(self, addr):
143
144         data = [0x30, addr]
145         data += self._crc(data)
146         (stat, recv, _) = self._tocard(0x0C, data)
147         return recv if stat == self.OK else None
148
149     def write(self, addr, data):
150
151         buf = [0x00, addr]
152         buf += self._crc(buf)
153         (stat, recv, bits) = self._tocard(0x0C, buf)
154
155         if not (stat == self.OK) or not (bits == 4) or not ((recv[0] & 0x0F) == 0x0A):
156             stat = self.ERR
157         else:
158             buf = []
159             for i in range(16):
160                 buf.append(data[i])
161             buf += self._crc(buf)
162             (stat, recv, bits) = self._tocard(0x0C, buf)
163             if not (stat == self.OK) or not (bits == 4) or not ((recv[0] & 0x0F) == 0x0A):
164                 stat = self.ERR
165
166         return stat

```

APPENDIX B: WEB DASHBOARD CODING

Index.html

```
1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <title>Smart School Bus Dashboard</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8
9 <body>
10
11   <div class="container">
12
13     <h1>Smart School Bus Dashboard</h1>
14
15     <!-- Environment Monitoring -->
16     <div class="card">
17       <h2>Environment Monitoring</h2>
18       <p id="temp">Temperature: --</p>
19       <p id="hum">Humidity: --</p>
20       <p id="gas">Gas Level: --</p>
21     </div>
22
23     <!-- Attendance Monitoring -->
24     <div class="card">
25       <h2>Attendance Monitoring</h2>
26
27       <p id="route">Route: --</p>
28       <p id="studentid">Student ID: --</p>
29       <p id="studentname">Student Name: --</p>
30       <p id="status_code">Status: --</p>
31     </div>
32
33     <!-- Ride Summary -->
34     <div class="card">
35       <h2>Ride Summary</h2>
36
37       <p id="ridestatus">Ride: --</p>
38       <p id="rideRoute">Route: --</p>
39       <p id="rideTotal">Total Students: --</p>
40       <hr>
41       <p id="ridelist">Students List:<br>--</p>
42     </div>
43
44   </div>
45
46   <script src="script.js"></script>
47
48 </body>
49
50 </html>
```

Script.js

```
1 // ===== CHANNEL INFO =====
2
3 const envChannelID = "2495344";
4 const envFeedAPI = "C3853QZPF802LXN";
5
6 const attChannelID = "2489892";
7 const attFeedAPI = "XMA4R3K732XZTVN";
8
9 // ===== STUDENTS =====
10
11 const studentNames = {
12   "1": "Muhammad Ali",
13   "201": "Jeyakumar Saravanan",
14   "251": "Joseph Vijay",
15   "258": "Sri",
16   "322": "Adriana Sarah Binti Ahmad Faiza"
17 };
18
19 // ===== LABELS =====
20
21 function statusLabel(code) {
22   switch (parseInt(code)) {
23     case 1: return "Present";
24     case 2: return "Duplicate Entry";
25     case 3: return "Unauthorized";
26     case 9: return "Ride ended";
27     default: return "--";
28   }
29 }
30
31 function routeLabel(routeNum) {
32   switch (parseInt(routeNum)) {
33     case 1: return "Home to School";
34     case 2: return "School to Home";
35     default: return "--";
36   }
37 }
38
39 // ===== MAIN FUNCTION =====
40
41 async function loadData() {
42
43   try {
44     // Fetch Environment Channel Data
45     let envResponse = await fetch(
46       `https://api.thingspeak.com/channels/${envChannelID}/feeds.json?api_key=${envFeedAPI}&results=20`
47     );
48     let envData = await envResponse.json();
49     let envFeeds = envData.feeds || [];
50
51     // Fetch Attendance Channel Data
52     let attResponse = await fetch(
53       `https://api.thingspeak.com/channels/${attChannelID}/feeds.json?api_key=${attFeedAPI}&results=20`
54     );
55     let attData = await attResponse.json();
56     let attFeeds = attData.feeds || [];
57
58   } catch {
59     // Error handling
60   }
61 }
```

```
57
58 // ===== ENVIRONMENT PROCESSING =====
59 let envFeed = [...envFeeds]
60   .reverse()
61   .find(f => f.field1 !== null);
62
63 if (envFeed) {
64   document.getElementById("temp").innerHTML =
65     `Temperature: ` + envFeed.field1 + " °C";
66   document.getElementById("hum").innerHTML =
67     `Humidity: ` + envFeed.field2 + " %";
68   document.getElementById("gas").innerHTML =
69     `Gas Level: ` + envFeed.field3;
70 } else {
71   document.getElementById("temp").innerHTML = `Temperature: --`;
72   document.getElementById("hum").innerHTML = `Humidity: --`;
73   document.getElementById("gas").innerHTML = `Gas Level: --`;
74 }
75
76 // ===== ATTENDANCE PROCESSING =====
77 let attendanceFeed = [...attFeeds]
78   .reverse()
79   .find(f => f.field4 !== null);
80
81 if (attendanceFeed) {
82   document.getElementById("route").innerHTML =
83     `Route: ` + routeLabel(attendanceFeed.field5);
84   let studentID = attendanceFeed.field4 || "--";
85   let studentName = studentNames[studentID] || "--";
86   document.getElementById("studentid").innerHTML =
87     `Student ID: ` + studentID;
88   document.getElementById("studentname").innerHTML =
89     `Student Name: ` + studentName;
90   document.getElementById("status_code").innerHTML =
91     `Status: ` + statusLabel(attendanceFeed.field6);
92 } else {
93   document.getElementById("route").innerHTML = `Route: --`;
94   document.getElementById("studentid").innerHTML = `Student ID: --`;
95   document.getElementById("studentname").innerHTML = `Student Name: --`;
96   document.getElementById("status_code").innerHTML = `Status: --`;
97 }
98
99 // ===== RIDE SUMMARY PROCESSING =====
100 // Find the absolute latest entry that contains a status code
101 let latestStatusFeed = [...attFeeds]
102   .reverse()
103   .find(f => f.field6 !== null);
104
105 // Display summary ONLY if the absolute latest state is "Ride Ended" (code 9)
106 if (latestStatusFeed && parseInt(latestStatusFeed.field6) === 9) {
107
108 }
```

```

113     document.getElementById("rideStatus").innerHTML =
114       "Ride: Ride Ended 🏁";
115
116     document.getElementById("rideRoute").innerHTML =
117       "Route: " + routeLabel(latestStatusFeed.field5);
118
119     document.getElementById("rideTotal").innerHTML =
120       "Total Students: " + (latestStatusFeed.field7 || "0");
121
122     // CLEAR HISTORY: Isolate Feeds belonging only to this specific trip
123     // 1. Find the index where this trip ended
124     let endIdx = attFeeds.indexOf(latestStatusFeed);
125
126     // 2. Look backwards from the end index to find where the previous ride ended (if any)
127     let startIdx = 0;
128     for (let i = endIdx - 1; i >= 0; i--) {
129       if (parseInt(attFeeds[i].field6) === 9) {
130         startIdx = i + 1; // The current trip starts right after the old trip's end
131         break;
132       }
133     }
134
135     // 3. Extract only the entries within this specific trip's window
136     let currentTripFeeds = attFeeds.slice(startIdx, endIdx + 1);
137
138     // Filter unique present students from the current trip window only
139     let presentStudents = [...new Set(
140       currentTripFeeds
141         .filter(f => parseInt(f.field6) === 1 && f.field4)
142         .map(f => f.field4)
143     )];
144
145     let list = presentStudents
146       .map(id => `${id} - ${studentNames[id] || "Unknown"}`)
147       .join("<br>");
148
149     document.getElementById("rideList").innerHTML =
150       "Present Students:<br>" + (list || "-.");
151   } else {
152     // Hide everything if button C hasn't been pressed or a new ride has started
153     document.getElementById("rideStatus").innerHTML = "Ride: --";
154     document.getElementById("rideRoute").innerHTML = "Route: --";
155     document.getElementById("rideTotal").innerHTML = "Total Students: --";
156     document.getElementById("rideList").innerHTML = "Present Students:<br>--";
157   }
158 }
159
160 } catch (error) {
161   console.log("Error loading data:", error);
162 }
163 }
164
165 // ===== START =====
166
167 loadData();
168 setInterval(loadData, 1000);

```

Style.css

```

1  body {
2
3    font-family: Arial, sans-serif;
4    background-color: #f4f4f4;
5    margin: 0;
6    padding: 20px;
7  }
8
9  .container {
10
11    max-width: 800px;
12    margin: auto;
13  }
14
15  h1 {
16
17    text-align: center;
18    color: #333;
19  }
20
21  .card {
22
23    background: white;
24    padding: 20px;
25    margin-top: 20px;
26
27    border-radius: 10px;
28
29    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
30  }
31
32  .card h2 {
33
34    color: #0066cc;
35  }
36
37  .card p {
38
39    font-size: 20px;
40    margin: 10px 0;
41  }

```

APPENDIX C: SOFTWARE PLATFORMS

ThingSpeak

My Channels

New Channel

Search by tag

Name	Created	Updated
Smart_School_Bus_System	2026-06-11	2026-06-27 19:32
Smart_School_Bus_System_Attendance	2026-06-16	2026-06-27 19:32

Help

Collect data in a ThingSpeak channel from a device, from another channel, or from the web.

Click **New Channel** to create a new ThingSpeak channel.

Click on the column headers of the table to sort by the entries in that column or click on a tag to show channels with that tag.

Learn to [create channels](#), explore and transform data.

Learn more about [ThingSpeak Channels](#).

Examples

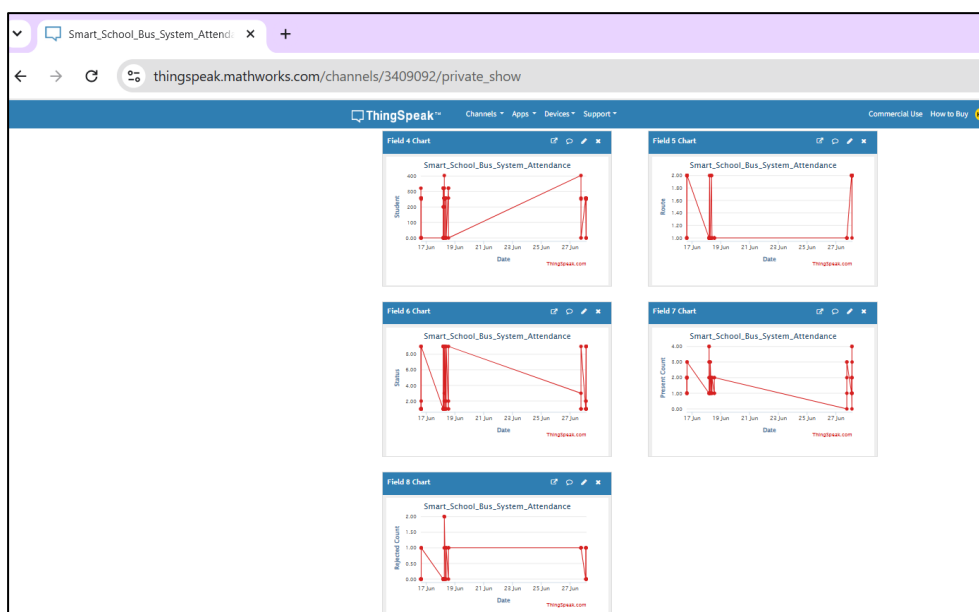
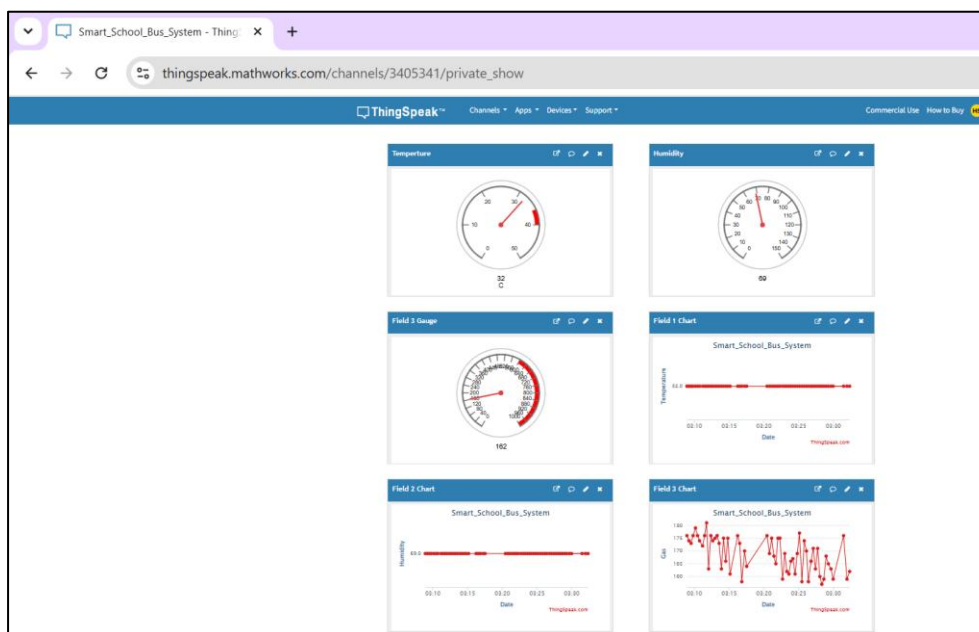
- [Arduino](#)
- [Arduino MKR1000](#)
- [ESP8266](#)
- [Raspberry Pi](#)

Upgrade

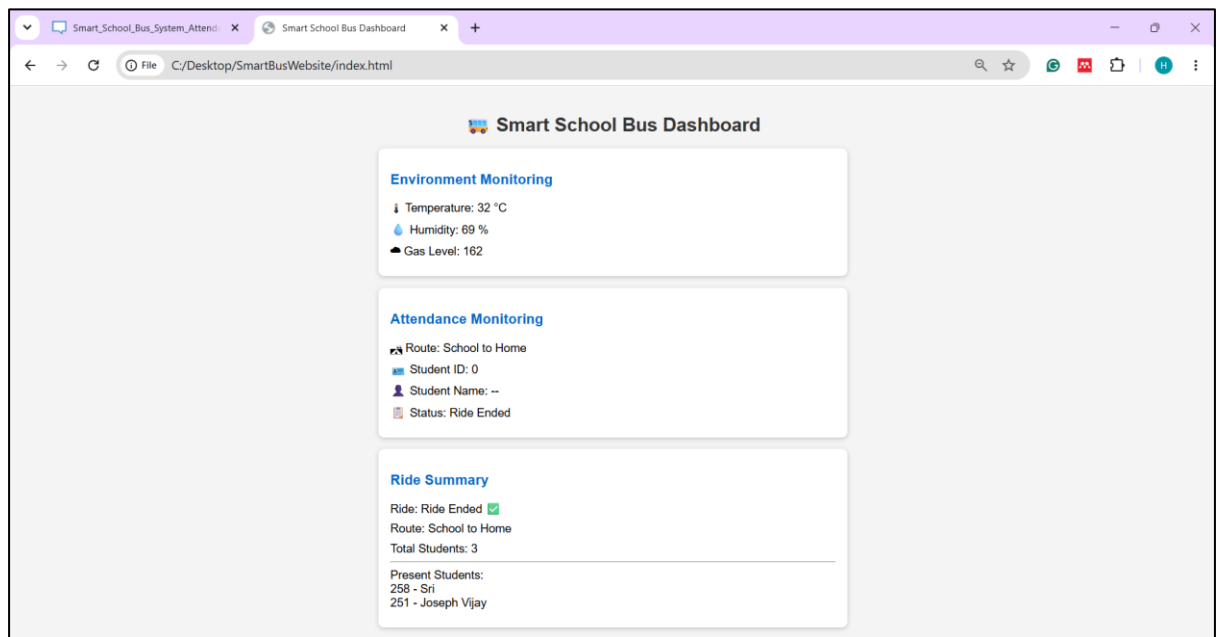
Need to send more data faster?

Need to use ThingSpeak for a commercial project?

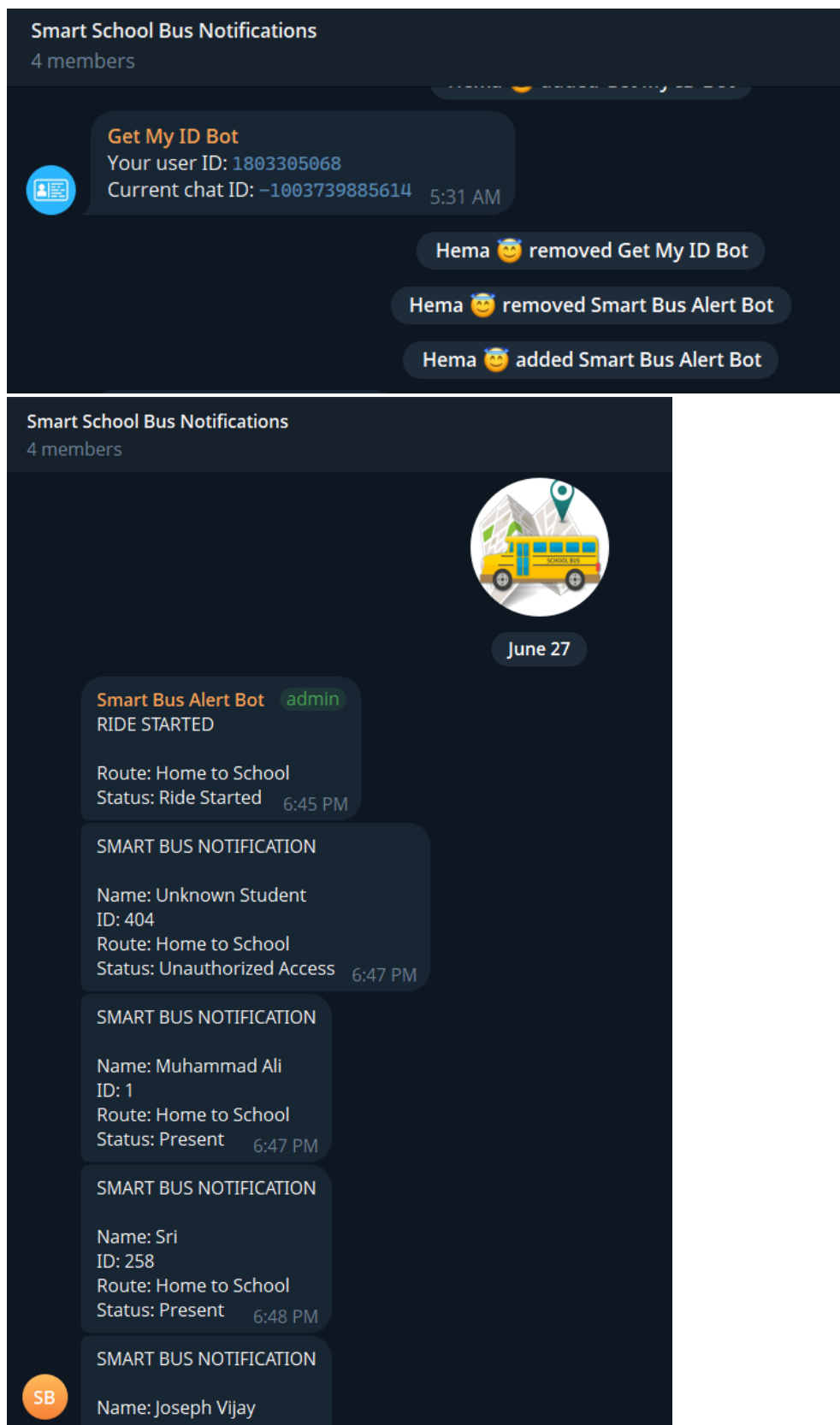
[Upgrade](#)



Web Dashboard



Telegram



APPENDIX D: GANTT CHART

GANTT CHART: HEMA A/P SAUNDARASAN (52224123036)					
Project Name:	Smart School Bus System with Automated Attendance and Environmental Monitoring				
Project Start:	15/10/2025				
Current Date :	21/1/2026				
Week in Progress:	Wk 14				
TASK	CHAPTER	DATE	TIME	APPROVED BY SUPERVISOR	PROGRESS
WEEK 1					
15/10/2025					
FYP Course Briefing	-	15/10/2025		Approved	100%
Approaching Supervisor	-	15/10/2025		Approved	100%
Supervisor confirmed: Madam Faridah	-	15/10/2025		Approved	100%
Finding Project Ideas	1	15/10/2025		Approved	100%
Set a meeting with the supervisor	1	16/10/2025	11:00:00 AM	Approved	100%
Idea Discussion & Approval	1	16/10/2025		Approved	100%
WEEK 2					
22/10/2025					
Preparing Proposal	1	22/10/2025		Approved	100%
Preparing Progress Report	1	23/10/2025		Approved	100%
Preparing Gantt Chart	1	24/10/2025		Approved	100%
Approval Proposal and Progress Report	1	25/10/2025		Approved	100%
WEEK 3					
26/10/2025					
Meeting with supervisor	1	27/10/2025	12:00:00 PM	Approved	100%
Approval for Gantt Chart	1	28/10/2025		Approved	100%
Corrections	1	29/10/2025		Approved	100%
WEEK 4					
30/10/2025					
Brainstorm idea	1	31/10/2025		Approved	100%
Started writing Chapter 1	1	1/11/2025		Approved	100%
Prepare list of requirements	1	2/11/2025		Approved	100%
WEEK 5					
3/11/2025					
Online meeting with supervisor	2	5/11/2025	11:30:00 PM	Approved	100%
Chapter 1	2	6/11/2025		Approved	100%
Update progress report	2	7/11/2025		Approved	100%
WEEK 6					
8/11/2025					
Started writing Chapter 2	2	9/11/2025		Approved	100%
Drafting chapter 2	2	11/11/2025		Approved	100%
Review flow chart design	2	13/11/2025		Approved	100%
WEEK 7					
13/11/2025					
Started writing Chapter 3	3	14/11/2025		Approved	100%
Described hardware components	3	20/11/2025		Approved	100%
Started preparing diagrams	3	21/11/2025		Approved	100%
Methodology part completion	3	25/11/2025		Approved	100%
Prepared costing table	3	26/11/2025		Approved	100%
Meeting with supervisor	3	27/11/2025	11:00:00 AM	Approved	100%
WEEK 8					
28/11/2025					
Rebuilt chapter 2	2	28/11/2025		Approved	100%
Rebuilt chapter 2	2	20/11/2025		Approved	100%
Rebuilt chapter 2	2	21/11/2025		Approved	100%
Rebuilt chapter 2	2	22/11/2025		Approved	100%
WEEK 9					
23/11/2025					
Rebuilt chapter 2	2	24/11/2025		Approved	100%
Rebuilt chapter 2	2	25/11/2025	12:00:00 PM	Approved	100%
Rebuilt chapter 2	2	26/11/2025		Approved	100%
Rebuilt chapter 2	2	27/11/2025		Approved	100%
WEEK 10					
28/11/2025					
Rebuilt chapter 2	2	29/11/2025		Approved	100%
Rewriting Chapter 3	3	30/11/2025		Approved	100%
List software & tools (Arduino IDE, etc.)	3	1/12/2025		Approved	100%
Brainstorm idea	3	2/12/2025		Approved	100%
WEEK 11					
3/12/2025					
Rewriting Chapter 3	3	5/12/2025		Approved	100%
Rewriting Chapter 3	3	8/12/2025		Approved	100%
Finalize circuit diagram	3	8/12/2025		Approved	100%
Rewriting Chapter 3	3	7/12/2025		Approved	100%
WEEK 12					
8/12/2025					
Rewriting Chapter 3	3	9/12/2025		Approved	100%
Meeting with supervisor	3	11/12/2025	1:00:00 PM	Approved	100%
Created simulation	3	13/12/2025		Approved	100%
Test initial connections in simulation	3	13/12/2025		Approved	100%
WEEK 13					
14/12/2025					
Run simulation testing	3	14/12/2025		Approved	100%